

DevOps & AWS Complete Learning Guide

1 DevOps & AWS Learning Hub

[1.1 Download PDFs](#)

[1.2 Suggested study pace](#)

[1.3 Page convention](#)

2 DevOps Curriculum: Tools & Concepts (Beginner → Advanced)

[2.1 Level 0 — Prerequisites \(before DevOps\)](#)

[2.1.1 Concepts](#)

[2.1.2 Tools & Skills](#)

[2.2 Level 1 — DevOps Foundations](#)

[2.2.1 Concepts](#)

[2.2.2 Tools](#)

[2.3 Level 2 — Infrastructure & Automation](#)

[2.3.1 Concepts](#)

[2.3.2 Tools](#)

[2.4 Level 3 — Containers & Orchestration](#)

[2.4.1 Concepts](#)

[2.4.2 Tools](#)

[2.5 Level 4 — CI/CD Pipelines](#)

[2.5.1 Concepts](#)

[2.5.2 Tools](#)

[2.6 Level 5 — Cloud Platforms \(cross-cutting\)](#)

[2.6.1 Concepts](#)

[2.6.2 Tools \(by provider\)](#)

[2.7 Level 6 — Observability & Reliability](#)

[2.7.1 Concepts](#)

[2.7.2 Tools](#)

[2.8 Level 7 — Security \(DevSecOps\)](#)

[2.8.1 Concepts](#)

[2.8.2 Tools](#)

[2.9 Level 8 — Data, Messaging & Databases \(Ops view\)](#)

[2.9.1 Concepts](#)

[2.9.2 Tools](#)

[2.10 Level 9 — Platform Engineering & Internal Developer Platforms \(adv\)](#)

[2.10.1 Concepts](#)

[2.10.2 Tools](#)

[2.11 Level 10 — Advanced & Specialized \(adv\)](#)

[2.11.1 Concepts](#)

[2.11.2 Tools](#)

[2.12 Certifications \(optional milestones\)](#)

[2.13 Learning project ideas \(apply everything\)](#)

3 AWS Cloud Curriculum: Services & Tools (Beginner → Advanced)

[3.1 1. Account, Identity & Governance ★](#)

[3.1.1 Concepts](#)

[3.2 2. Networking & Content Delivery ★](#)

[3.2.1 Tools](#)

[3.3 3. Compute ★](#)

[3.3.1 Tools](#)

[3.4 4. Storage ★](#)

[3.4.1 Concepts](#)

[3.5 5. Databases ★](#)

[3.6 6. Application Integration & Messaging ★](#)

[3.7 7. Developer Tools & CI/CD ★](#)

[3.8 8. Observability & Management ★](#)

[3.9 9. Security, Identity & Compliance ★](#)

[3.10 10. Migration & Transfer ♦](#)

[3.11 11. Analytics & Big Data ♦](#)

[3.12 12. Machine Learning & AI ◇](#)

[3.13 13. IoT, Media & Specialized ◇](#)

[3.14 14. Cost & Billing ★](#)

[3.14.1 Concepts](#)

[3.15 15. Well-Architected Framework \(conceptual ★\)](#)

[3.16 Certification path \(recommended order\)](#)

[3.17 Hands-on lab progression](#)

[4 DevOps Handbook — Master Table of Contents](#)

[4.1 How to read this handbook](#)

[4.2 Part I — Introduction & Mindset \(Pages 1–60\)](#)

[4.3 Part II — Linux & Shell Mastery \(Pages 61–140\)](#)

[4.4 Part III — Git & Collaboration \(Pages 141–200\)](#)

[4.5 Part IV — Networking for DevOps \(Pages 201–280\)](#)

[4.6 Part V — Scripting & Programming \(Pages 281–360\)](#)

[4.7 Part VI — Infrastructure as Code \(Pages 361–480\)](#)

[4.8 Part VII — Containers \(Pages 481–580\)](#)

[4.9 Part VIII — Kubernetes \(Pages 581–720\)](#)

[4.10 Part IX — CI/CD \(Pages 721–840\)](#)

[4.11 Part X — Observability & SRE \(Pages 841–960\)](#)

[4.12 Part XI — Security \(DevSecOps\) \(Pages 961–1080\)](#)

[4.13 Part XII — Advanced Topics \(Pages 1081–1200\)](#)

[4.14 Expansion status](#)

[5 Chapter 1: What Is DevOps? History and Culture](#)

[5.1 1.1 The problem DevOps solves](#)

[5.2 1.2 A working definition](#)

[5.3 1.3 Brief history](#)

[5.4 1.4 Culture before tools](#)

[5.5 1.5 DevOps practices \(overview\)](#)

[5.6 1.6 Organizational models](#)

[5.6.1 Functional silos \(anti-pattern for DevOps\)](#)

[5.6.2 Cross-functional product teams \(recommended\)](#)

[5.6.3 DevOps enablement / platform team](#)

[5.7 1.7 What DevOps is NOT](#)

[5.8 1.8 Your learning path in this handbook](#)

[5.9 1.9 Chapter summary](#)

[5.10 Lab 1.1 — Reflect on your current workflow](#)

[5.11 Review questions](#)

[6 Chapter 2: CALMS, Three Ways, and Team Topologies](#)

[6.1 2.1 The CALMS framework](#)

[6.1.1 Culture \(C\)](#)

[6.1.2 Automation \(A\)](#)

[6.1.3 Lean \(L\)](#)

[6.1.4 Measurement \(M\)](#)

[6.1.5 Sharing \(S\)](#)

[6.2 2.2 The Three Ways \(Gene Kim\)](#)

[6.2.1 First Way: Flow \(left to right\)](#)

[6.2.2 Second Way: Feedback \(right to left\)](#)

[6.2.3 Third Way: Continuous learning and experimentation](#)

[6.3 2.3 Team Topologies \(Skelton & Pais\)](#)

[6.3.1 Interaction modes](#)

[6.4 2.4 Conway's Law and reverse Conway maneuver](#)

[6.5 2.5 Maturity model \(informal\)](#)

[6.6 2.6 Chapter summary](#)

[6.7 Lab 2.1 — CALMS assessment](#)

[6.8 Review questions](#)

[7 Chapter 3: DevOps vs SRE vs Platform Engineering](#)

[7.1 3.1 Three related disciplines](#)

[7.2 3.2 Site Reliability Engineering \(SRE\)](#)

[7.2.1 Core SRE principles](#)

[7.2.2 SRE team responsibilities](#)

[7.2.3 Error budget example](#)

[7.3 3.3 Platform Engineering](#)

[7.3.1 What platform teams build](#)

[7.3.2 Platform vs traditional ops](#)

[7.4 3.4 How they fit together](#)

[7.5 3.5 Job titles in the market](#)

[7.6 3.6 Chapter summary](#)

[7.7 Lab 3.1](#)

[8 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions](#)

[8.1 5.1 Why Linux dominates DevOps](#)

[8.2 5.2 Filesystem hierarchy \(FHS\)](#)

[8.3 5.3 Essential navigation commands](#)

[8.4 5.4 File operations](#)

[8.4.1 Finding files](#)

[8.5 5.5 Users, groups, and permissions](#)

[8.5.1 Users and groups](#)

[8.5.2 Permission model](#)

[8.5.3 Special permissions](#)

[8.6 5.6 Package management \(Debian/Ubuntu\)](#)

[8.7 5.7 Text processing \(daily DevOps tools\)](#)

[8.8 5.8 Environment and shell](#)

[8.9 5.9 SSH — remote access](#)

[8.10 5.10 Chapter summary](#)

[8.11 Lab 5.1 — Linux sandbox](#)

[8.12 Review questions](#)

[9 Chapter 6: Processes, systemd, and Service Management](#)

[9.1 6.1 Understanding Linux processes](#)

[9.1.1 Process states](#)

[9.1.2 Signals](#)

[9.2 6.2 systemd — modern service manager](#)

[9.2.1 Unit files](#)

[9.3 6.3 Cron and timers](#)

[9.3.1 cron — scheduled tasks](#)

[9.3.2 systemd timers \(preferred on modern systems\)](#)

[9.4 6.4 Resource limits and troubleshooting](#)

[9.4.1 Common production issues](#)

[9.5 6.5 Chapter summary](#)

[9.6 Lab 6.1](#)

[10 Chapter 9: Git Internals — Objects, Refs, and the DAG](#)

[10.1 9.1 Git is a content-addressable filesystem](#)

[10.2 9.2 How a commit is stored](#)

[10.3 9.3 Essential Git commands \(beyond basics\)](#)

[10.3.1 Undoing mistakes](#)

[10.4 9.4 Branches and merging](#)

[10.4.1 Merge vs rebase](#)

[10.5 9.5 Remote repositories](#)

[10.5.1 Tracking branches](#)

[10.6 9.6 .gitignore and hooks](#)

[10.6.1 .gitignore](#)

[10.6.2 Git hooks \(`.git/hooks/` \)](#)

[10.7 9.7 Chapter summary](#)

[10.8 Lab 9.1](#)

[11 Chapter 20: IaC Principles — State, Idempotency, and Drift](#)

[11.1 20.1 What is Infrastructure as Code?](#)

[11.2 20.2 Declarative vs imperative](#)

[11.3 20.3 Idempotency](#)

[11.4 20.4 State management](#)

[11.4.1 State best practices](#)

[11.5 20.5 Configuration drift](#)

[11.6 20.6 IaC workflow in teams](#)

[11.6.1 Directory structure \(example\)](#)

[11.7 20.7 Testing IaC](#)

[11.8 20.8 Chapter summary](#)

[11.9 Lab 20.1](#)

[12 Chapter 21: Terraform — HCL, Providers, and Modules](#)

[12.1 21.1 Terraform overview](#)

[12.2 21.2 Basic configuration](#)

[12.3 21.3 Resources, data sources, and dependencies](#)

[12.3.1 Data sources — read existing infrastructure](#)

[12.3.2 Implicit vs explicit dependencies](#)

[12.4 21.4 Modules — reusable infrastructure](#)

[12.5 21.5 Lifecycle meta-arguments](#)

[12.6 21.6 Workspaces and environments](#)

[12.7 21.7 CI/CD integration](#)

[12.8 21.8 Chapter summary](#)

[12.9 Lab 21.1](#)

[13 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds](#)

[13.1 26.1 Containers in one paragraph](#)

[13.2 26.2 Core concepts](#)

[13.3 26.3 Essential commands](#)

[13.4 26.4 Dockerfile best practices](#)

[13.4.1 Best practices checklist](#)

[13.5 26.5 Docker Compose for local development](#)

[13.6 26.6 Security basics](#)

[13.7 26.7 Chapter summary](#)

[13.8 Lab 26.1](#)

[14 Chapter 37: CI/CD Concepts and Pipeline Design](#)

[14.1 37.1 Continuous Integration \(CI\)](#)

[14.2 37.2 Continuous Delivery vs Continuous Deployment](#)

[14.3 37.3 Pipeline stages](#)

[14.4 37.4 GitHub Actions example](#)

[14.5 37.5 Branching and trunk-based development](#)

[14.6 37.6 Deployment strategies](#)

[14.7 37.7 Pipeline secrets and artifacts](#)

[14.8 37.8 DORA metrics in your pipeline](#)

[14.9 37.9 Chapter summary](#)

[14.10 Lab 37.1](#)

[15 AWS Handbook — Master Table of Contents](#)

[15.1 Part I — Cloud Foundations \(Pages 1-80\)](#)

[15.2 Part II — Identity & Access Management \(Pages 81-160\)](#)

[15.3 Part III — Networking \(Pages 161-280\)](#)

[15.4 Part IV — Compute \(Pages 281-400\)](#)

[15.5 Part V — Storage \(Pages 401-480\)](#)

[15.6 Part VI — Databases \(Pages 481-560\)](#)

[15.7 Part VII — Application Integration \(Pages 561-620\)](#)

[15.8 Part VIII — DevOps on AWS \(Pages 621-700\)](#)

[15.9 Part IX — Observability & Operations \(Pages 701-760\)](#)

[15.10 Part X — Security & Compliance \(Pages 761-800\)](#)

[15.11 Expansion status](#)

[16 Chapter 1: Introduction to AWS & the Shared Responsibility Model](#)

[16.1 1.1 What is Amazon Web Services \(AWS\)?](#)

[16.1.1 Why AWS matters for DevOps](#)

[16.2 1.2 Cloud service models](#)

[16.3 1.3 The Shared Responsibility Model](#)

[16.3.1 AWS is responsible for **Security OF the Cloud**](#)

[16.3.2 You are responsible for **Security IN the Cloud**](#)

[16.3.3 Example: RDS vs EC2 MySQL](#)

[16.4 1.4 Core AWS concepts](#)

[16.5 1.5 AWS console, CLI, and IaC](#)

[16.6 1.6 Pricing fundamentals](#)

[16.7 1.7 Well-Architected Framework \(preview\)](#)

[16.8 1.8 Chapter summary](#)

[16.9 Lab 1.1 — Create your AWS account](#)

[16.10 Review questions](#)

[17 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge](#)

[17.1 2.1 Global infrastructure overview](#)

[17.2 2.2 Regions](#)

[17.2.1 Choosing a region](#)

[17.2.2 Regional vs global services](#)

[17.3 2.3 Availability Zones \(AZs\)](#)

[17.3.1 Multi-AZ patterns](#)

[17.4 2.4 Local Zones, Wavelength, and Outposts](#)

[17.5 2.5 Edge locations and CloudFront](#)

[17.6 2.6 Fault isolation design principles](#)

[17.6.1 S3 durability vs availability](#)

[17.7 2.7 Service quotas \(limits\)](#)

[17.8 2.8 Chapter summary](#)

[17.9 Lab 2.1 — Explore regions in CLI](#)

[17.10 Review questions](#)

[18 Chapter 3: AWS Account Setup, Billing, and Cost Basics](#)

[18.1 3.1 Creating a secure AWS account](#)

[18.1.1 Step-by-step](#)

[18.1.2 Root account checklist](#)

[18.2 3.2 AWS Free Tier](#)

[18.3 3.3 Billing and Cost Management](#)

[18.3.1 Budget example](#)

[18.4 3.4 Cost optimization fundamentals](#)

[18.5 3.4 Consolidated billing with Organizations](#)

[18.6 3.5 Support plans \(overview\)](#)

[18.7 3.6 Chapter summary](#)

[18.8 Lab 3.1](#)

[19 Chapter 5: IAM Users, Groups, Roles, and Policies](#)

[19.1 5.1 What is IAM?](#)

[19.2 5.2 IAM identity types](#)

[19.3 5.3 Policies — the core of permissions](#)

[19.3.1 Policy structure](#)

[19.3.2 Policy types](#)

[19.4 5.4 Users and groups](#)

[19.4.1 Create a developer group \(example workflow\)](#)

[19.5 5.5 Roles and trust policies](#)

[19.5.1 EC2 instance role example](#)

[19.5.2 Assume role \(cross-account or CLI\)](#)

[19.6 5.6 AWS managed vs customer managed policies](#)

[19.7 5.7 Least privilege in practice](#)

[19.7.1 Tag-based access \(ABAC\)](#)

[19.8 5.8 Common IAM actions for DevOps](#)

[19.9 5.9 IAM Access Analyzer & CloudTrail](#)

[19.10 5.10 Chapter summary](#)

[19.11 Lab 5.1 — IAM hands-on](#)

[19.12 Review questions](#)

[20 Chapter 6: IAM Best Practices and Permission Boundaries](#)

[20.1 6.1 The IAM security baseline](#)

[20.2 6.2 Prefer roles over users for applications](#)

[20.2.1 CI/CD role pattern](#)

[20.3 6.3 Permission boundaries](#)

[20.4 6.4 Policy writing tips](#)

[20.4.1 Start with AWS managed, refine to customer managed](#)

[20.4.2 Use IAM Policy Simulator](#)

[20.4.3 Avoid these anti-patterns](#)

[20.5 6.5 Service-linked roles](#)

[20.6 6.6 Cross-account access](#)

[20.7 6.7 Chapter summary](#)

[20.8 Lab 6.1](#)

[21 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables](#)

[21.1 9.1 What is a VPC?](#)

[21.2 9.2 CIDR and IP planning](#)

[21.2.1 Example VPC design](#)

[21.3 9.3 Subnets](#)

[21.4 9.4 Route tables](#)

[21.5 9.5 Internet Gateway and NAT \(preview\)](#)

[21.6 9.6 Security Groups vs NACLs](#)

[21.7 9.7 Terraform VPC snippet](#)

[21.8 9.8 Chapter summary](#)

[21.9 Lab 9.1](#)

[22 Chapter 10: Internet Gateway, NAT Gateway, and Routing](#)

[22.1 10.1 Internet Gateway \(IGW\)](#)

[22.2 10.2 NAT Gateway](#)

[22.2.1 NAT Gateway facts](#)

[22.3 10.3 VPC Endpoints — avoid NAT costs](#)

[22.4 10.4 Elastic IP \(EIP\)](#)

[22.5 10.5 Troubleshooting connectivity](#)

[22.5.1 Useful commands](#)

[22.6 10.6 High availability networking](#)

[22.7 10.7 Chapter summary](#)

[22.8 Lab 10.1](#)

[23 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates](#)

[23.1 15.1 Amazon EC2 overview](#)

[23.2 15.2 Instance families](#)

[23.2.1 Purchasing options](#)

[23.3 15.3 AMIs and launch templates](#)

[23.4 15.4 User data — bootstrap scripts](#)

[23.5 15.5 Auto Scaling Groups \(ASG\)](#)

[23.6 15.6 EC2 best practices](#)

[23.7 15.7 Chapter summary](#)

[23.8 Lab 15.1](#)

[24 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle](#)

[24.1 20.1 Amazon S3 overview](#)

[24.2 20.2 Core concepts](#)

[24.3 20.3 Storage classes](#)

[24.4 20.4 Versioning and lifecycle](#)

[24.4.1 Versioning](#)

[24.4.2 Lifecycle rules](#)

[24.5 20.5 Security](#)

[24.5.1 Block Public Access \(account and bucket level\)](#)

[24.5.2 Encryption](#)

[24.5.3 Bucket policies vs ACLs](#)

[24.6 20.6 Static website hosting](#)

[24.7 20.7 S3 performance tips](#)

[24.8 20.8 Chapter summary](#)

[24.9 Lab 20.1](#)

[25 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards](#)

[25.1 35.1 Amazon CloudWatch overview](#)

[25.2 35.2 Metrics](#)

[25.2.1 View metrics](#)

[25.2.2 Custom metrics](#)

[25.3 35.3 Alarms](#)

[25.3.1 Alarm states](#)

[25.3.2 Composite alarms](#)

[25.4 35.4 Dashboards](#)

[25.5 35.5 CloudWatch Logs \(overview\)](#)

[25.6 35.6 Observability best practices on AWS](#)

[25.7 35.7 Chapter summary](#)

[25.8 Lab 35.1](#)

1 DevOps & AWS Learning Hub

A structured path from zero to advanced, plus two handbook series:

Resource	Target size	Status
DevOps Curriculum	Reference list	Complete
AWS Curriculum	Reference list	Complete
DevOps Handbook	~1,200 pages	TOC + Part I-II written; Parts III-XII outlined
AWS Handbook	~800 pages	TOC + Part I-II written; Parts III-X outlined

1.1 Download PDFs

Pre-built PDFs are in the [pdf/](#) folder:

PDF	Description
devops-handbook.pdf	DevOps handbook — 11 chapters + full TOC
aws-handbook.pdf	AWS handbook — 11 chapters + full TOC
devops-curriculum.pdf	DevOps tools & concepts reference
aws-curriculum.pdf	AWS services reference
devops-aws-complete-guide.pdf	All content in one PDF

Rebuild PDFs after editing markdown:

```
./scripts/build-pdfs.sh
```

Requires `pandoc` and `wkhtmltopdf`.

1. **Start with the curricula** — they list every tool, concept, and AWS service in learning order.
2. **Follow the handbook TOCs** — each chapter has a page budget; work through sequentially.
3. **Expand on demand** — ask an AI assistant or instructor to flesh out any chapter number from the TOC.

1.2 Suggested study pace

Phase	Duration (self-paced)	Focus
Foundations	4-8 weeks	Linux, Git, networking, scripting
Core DevOps	8-12 weeks	CI/CD, containers, IaC, cloud basics
Intermediate	8-12 weeks	Kubernetes, observability, security
		CSP platform

Advanced	12+ weeks	SRE, platform engineering, multi-cloud, FinOps
----------	-----------	--

1.3 Page convention

Each handbook “page” $\approx$ 400 words of instructional content + diagrams/commands. Labs and exercises add extra pages in the TOC estimates.

2 DevOps Curriculum: Tools & Concepts (Beginner → Advanced)

Complete reference of what to learn, in recommended order. Items marked **(adv)** are advanced topics.

2.1 Level 0 — Prerequisites (before DevOps)

2.1.1 Concepts

- Software development lifecycle (SDLC)
- Agile, Scrum, Kanban
- Waterfall vs iterative delivery
- Client-server architecture
- APIs (REST, GraphQL basics)
- Databases (SQL vs NoSQL overview)
- Environment tiers: dev, staging, production

2.1.2 Tools & Skills

Category	Tools / Skills
Operating systems	Linux (Ubuntu, RHEL), Windows Server basics
Shell	Bash, Zsh, PowerShell basics
Text editing	Vim, Nano, VS Code
Networking	TCP/IP, DNS, HTTP/HTTPS, TLS, ports, firewalls
Version control	Git (clone, branch, merge, rebase, cherry-pick)
Scripting	Python, Bash, Go (pick one primary)
Documentation	Markdown, diagrams (draw.io, Mermaid)

2.2 Level 1 — DevOps Foundations

2.2.1 Concepts

- What is DevOps? Culture, CALMS (Culture, Automation, Lean, Measurement, Sharing)
- DevOps vs SysAdmin vs SRE vs Platform Engineering
- Continuous Integration (CI) vs Continuous Delivery (CD) vs Continuous Deployment
- Build artifacts, immutable infrastructure
- Configuration vs secrets
- The Twelve-Factor App
- DevSecOps introduction
- GitOps introduction

2.2.2 Tools

Category	Tools
Git hosting	GitHub, GitLab, Bitbucket, Azure DevOps Repos
SSH & keys	OpenSSH, ssh-agent, key pairs

Package managers	apt, yum/dnf, Homebrew
Build tools	Make, Maven, Gradle, npm/yarn/pnpm
Virtualization	VirtualBox, VMware, Hyper-V
Cloud CLI basics	AWS CLI, gcloud, az

2.3 Level 2 — Infrastructure & Automation

2.3.1 Concepts

- Infrastructure as Code (IaC) principles
- Idempotency, declarative vs imperative
- Server provisioning & bootstrapping
- Configuration management
- Immutable vs mutable infrastructure
- Golden images / AMIs
- Inventory & state management

2.3.2 Tools

Category	Tools
IaC — Terraform	Terraform, OpenTofu, Terragrunt
IaC — Cloud-native	AWS CloudFormation, Azure Bicep/ARM, Google Deployment Manager, Pulumi, CDK (AWS/Azure/GCP)
Configuration mgmt	Ansible, Chef, Puppet, SaltStack
Image building	Packer
Secrets (intro)	HashiCorp Vault, AWS Secrets Manager, SOPS
Cloud VMs	EC2, Azure VMs, GCE

2.4 Level 3 — Containers & Orchestration

2.4.1 Concepts

- Containers vs VMs
- Images, layers, registries
- Dockerfile best practices
- Container networking & storage
- Orchestration primitives (Pod, Service, Deployment)
- Service mesh concepts
- Sidecar pattern
- Helm charts & package management

2.4.2 Tools

Category	Tools
Runtime	Docker, containerd, CRI-O, Podman
Compose / local	Docker Compose, Dev Containers

Registries	Docker Hub, ECR, GCR, ACR, Harbor, Artifactory
Orchestration	Kubernetes (K8s), Amazon ECS, Amazon EKS, GKE, AKS
K8s tooling	kubectl, k9s, Lens, Helm, Kustomize
Service mesh	Istio, Linkerd, Consul Connect
Local K8s	minikube, kind, k3s, Docker Desktop K8s

2.5 Level 4 — CI/CD Pipelines

2.5.1 Concepts

- Pipeline stages: source → build → test → scan → deploy
- Branching strategies (GitFlow, trunk-based)
- Artifact versioning & promotion
- Blue/green, canary, rolling deployments
- Feature flags
- Pipeline as Code
- DORA metrics (deployment frequency, lead time, MTTR, change failure rate)

2.5.2 Tools

Category	Tools
CI/CD platforms	Jenkins, GitHub Actions, GitLab CI, CircleCI, Travis CI, Azure Pipelines, AWS CodePipeline, Tekton, Argo CD (GitOps)
Build agents	Jenkins agents, GitHub-hosted/self-hosted runners
Artifact repos	Nexus, Artifactory, GitHub Packages
GitOps	Argo CD, Flux, Fleet
Deployment	Spinnaker, Harness, Octopus Deploy

2.6 Level 5 — Cloud Platforms (cross-cutting)

2.6.1 Concepts

- Shared responsibility model
- Regions, AZs, edge locations
- IAM (users, roles, policies, least privilege)
- VPC networking (subnets, routing, NAT, peering)
- Load balancing (L4 vs L7)
- Object storage vs block vs file
- Managed vs self-managed services
- Well-Architected Framework pillars
- Cost optimization & tagging
- Multi-account / landing zone strategy

2.6.2 Tools (by provider)

Provider	Core services to learn
----------	------------------------

AWS	See curriculum-aws.md
Azure	Resource Manager, AAD, VNet, AKS, App Service
GCP	IAM, VPC, GKE, Cloud Run, Cloud Build

2.7 Level 6 — Observability & Reliability

2.7.1 Concepts

- Monitoring vs observability (metrics, logs, traces)
- SLI, SLO, SLA, error budgets
- Alerting philosophy (symptom vs cause)
- On-call, incident management, postmortems
- Chaos engineering
- Capacity planning
- Distributed tracing

2.7.2 Tools

Category	Tools
Metrics	Prometheus, Grafana, Datadog, New Relic, CloudWatch, Azure Monitor
Logging	ELK/EFK (Elasticsearch, Logstash/Fluentd, Kibana), Loki, Splunk, CloudWatch Logs
Tracing	Jaeger, Zipkin, OpenTelemetry, AWS X-Ray, Honeycomb
APM	Datadog APM, Dynatrace, AppDynamics
Incident mgmt	PagerDuty, Opsgenie, incident.io
Status	Statuspage
Chaos	Chaos Monkey, LitmusChaos, Gremlin

2.8 Level 7 — Security (DevSecOps)

2.8.1 Concepts

- Shift-left security
- OWASP Top 10
- Supply chain security (SBOM, signing)
- Zero trust networking
- CIS benchmarks
- Compliance frameworks (SOC 2, PCI-DSS, HIPAA overview)
- Threat modeling
- Principle of least privilege

2.8.2 Tools

Category	Tools
SAST	SonarQube, Semgrep, CodeQL, Checkmarx

DAST	OWASP ZAP, Burp Suite
Container scan	Trivy, Gype, Snyk, Clair
IaC scan	Checkov, tfsec, KICS, cfn-lint
Secrets scan	git-secrets, TruffleHog, Gitleaks
Policy as code	OPA, Gatekeeper, Kyverno, Sentinel
WAF / edge	AWS WAF, Cloudflare, ModSecurity
PKI / certs	cert-manager, Let's Encrypt, ACM
SIEM	Splunk, Elastic Security, AWS Security Hub

2.9 Level 8 — Data, Messaging & Databases (Ops view)

2.9.1 Concepts

- Backup & restore strategies (RPO/RTO)
- Replication, failover, DR
- Connection pooling
- Caching layers
- Message queues & event-driven architecture
- Database migrations in CI/CD

2.9.2 Tools

Category	Tools
RDBMS	PostgreSQL, MySQL, MariaDB, SQL Server
NoSQL	MongoDB, DynamoDB, Redis, Cassandra
Messaging	Kafka, RabbitMQ, SQS, SNS, Pub/Sub
Streaming	Kinesis, Flink, Spark Streaming
Backup	Velero (K8s), AWS Backup, pg_dump, Percona XtraBackup
Migrations	Flyway, Liquibase, Alembic

2.10 Level 9 — Platform Engineering & Internal Developer Platforms (adv)

2.10.1 Concepts

- Platform as a product
- Developer experience (DevEx)
- Self-service portals
- Golden paths & paved roads
- Service catalogs
- Backstage software templates

2.10.2 Tools

Category	Tools

IDP	Backstage, Port, Cortex
API gateways	Kong, Apigee, AWS API Gateway, Ambassador
Service discovery	Consul, Eureka, CoreDNS
Feature flags	LaunchDarkly, Unleash, Flagsmith

2.11 Level 10 — Advanced & Specialized (adv)

2.11.1 Concepts

- Site Reliability Engineering (SRE) — toil reduction, automation
- FinOps & cloud cost governance
- Multi-cloud & hybrid cloud
- Edge computing & CDN strategy
- WebAssembly in ops (adv)
- eBPF for observability (adv)
- ML/LLM ops (MLOps) pipelines

2.11.2 Tools

Category	Tools
FinOps	Kubecost, CloudHealth, AWS Cost Explorer, Infracost
CDN	CloudFront, Cloudflare, Fastly, Akamai
DNS	Route 53, Cloudflare DNS, ExternalDNS
Workflow	Airflow, Temporal, Step Functions
MLOps	MLflow, Kubeflow, SageMaker (AWS)
eBPF	Cilium, Pixie, Falco

2.12 Certifications (optional milestones)

Track	Certifications
Linux	LPIC, RHCSA/RHCE
Cloud	AWS SAA, AWS DVA, AWS SOA, AWS DOP, AWS SAP; Azure AZ-104, AZ-400; GCP PCA
Kubernetes	CKA, CKAD, CKS
Terraform	HashiCorp Terraform Associate
Security	CISSP (broad), AWS Security Specialty
SRE	Google SRE book + practice (no single cert)

2.13 Learning project ideas (apply everything)

1. **Static site pipeline** — S3 + CloudFront + GitHub Actions + Terraform

2. **Microservices on EKS** — Helm, Prometheus, Argo CD
3. **Full observability stack** — app + OpenTelemetry + Grafana stack
4. **Secure CI/CD** — SAST/DAST/SCA in pipeline, signed images
5. **Multi-env IaC** — dev/stage/prod with Terragrunt or Terraform workspaces
6. **Incident simulation** — chaos tests + runbooks + PagerDuty
7. **Internal developer portal** — Backstage with one golden-path service

3 AWS Cloud Curriculum: Services & Tools

(Beginner → Advanced)

All major AWS services grouped by domain, with learning priority.

Legend: ★ = core (learn first) · ◆ = intermediate · ◇ = advanced/specialized

3.1 1. Account, Identity & Governance ★

Service / Tool	Purpose
AWS Organizations ★	Multi-account structure, SCPs
IAM ★	Users, groups, roles, policies, MFA
IAM Identity Center (SSO) ★	Workforce federation, permission sets
AWS Control Tower ◆	Landing zone automation
AWS Config ◆	Resource compliance & history
AWS CloudTrail ★	API audit logging
AWS Service Catalog ◇	Governed self-service provisioning
AWS Resource Access Manager (RAM) ◆	Cross-account resource sharing
AWS Account Factory ◆	Automated account vending (Control Tower)
AWS CLI / CloudShell ★	Command-line & browser shell
AWS Management Console ★	Web UI
AWS CloudFormation ★	Infrastructure as Code (native)
AWS CDK ◆	IaC in TypeScript/Python/etc.
Terraform + AWS Provider ★	Industry-standard IaC
AWS Tagging & Resource Groups ★	Cost & ops organization

3.1.1 Concepts

- Shared responsibility model
- Root account hygiene (MFA, no daily use)
- Least privilege & permission boundaries
- SCP vs IAM policy
- Cross-account roles & assume-role
- AWS global vs regional services

3.2 2. Networking & Content Delivery ★

Service	Purpose
VPC ★	Virtual network, subnets, route tables
Internet Gateway / NAT Gateway ★	Public & private egress

VPC Endpoints (Gateway & Interface) ♦	Private AWS API access
Security Groups & NACLs ★	Stateful vs stateless firewalls
Elastic IP ★	Static public IPs
Route 53 ★	DNS, health checks, routing policies
CloudFront ★	CDN, edge caching
ALB / NLB / GWLB ★	Layer 7, 4, and gateway load balancers
VPC Peering / Transit Gateway ♦	Inter-VPC & hybrid connectivity
AWS VPN / Direct Connect ♦	On-prem hybrid networking
PrivateLink ♦	Private service exposure
AWS Global Accelerator ◇	Anycast for global apps
Route 53 Resolver ♦	Hybrid DNS
AWS Network Firewall ◇	Managed perimeter firewall
VPC Lattice ◇	Service-to-service networking

3.2.1 Tools

- **Reachability Analyzer, VPC Flow Logs, Traffic Mirroring**

3.3 3. Compute ★

Service	Purpose
EC2 ★	Virtual machines, AMIs, instance types
Auto Scaling Groups ★	Horizontal scaling
Elastic Beanstalk ♦	PaaS for web apps
Lambda ★	Serverless functions
ECS / Fargate ★	Container orchestration (AWS-native)
EKS ★	Managed Kubernetes
AWS Batch ♦	Batch/HPC jobs
Lightsail ♦	Simplified VPS
App Runner ♦	Container PaaS
EC2 Image Builder ♦	Automated AMI pipelines
AWS Outposts / Wavelength / Local Zones ◇	Edge & hybrid compute
Serverless Application Repository ◇	Share Lambda apps

3.3.1 Tools

- **AWS Systems Manager** (Session Manager, Run Command, Patch Manager)
- **EC2 Instance Connect**

3.4 4. Storage ★

Service	Purpose
S3 ★	Object storage, versioning, lifecycle
EBS ★	Block storage for EC2
EFS ◆	Managed NFS
FSx ◆	Windows/Lustre/ONTAP/OpenZFS file systems
S3 Glacier ◆	Archive tiers
Storage Gateway ◇	Hybrid storage
AWS Backup ◆	Centralized backup
DataSync / Transfer Family ◆	Large data migration

3.4.1 Concepts

- Storage classes (S3 Standard, IA, Glacier)
- Encryption at rest (SSE-S3, SSE-KMS, SSE-C)
- S3 bucket policies vs ACLs
- EBS volume types (gp3, io2, st1)

3.5 5. Databases ★

Service	Purpose
RDS ★	Managed MySQL, PostgreSQL, MariaDB, Oracle, SQL Server
Aurora ★	AWS-native relational, serverless option
DynamoDB ★	Serverless NoSQL key-value/document
ElastiCache ◆	Redis / Memcached
DocumentDB ◆	MongoDB-compatible
Neptune ◇	Graph database
Timestream ◇	Time-series
Keyspaces ◇	Cassandra-compatible
Redshift ◆	Data warehouse
QLDB ◇	Ledger database
MemoryDB for Redis ◆	Durable Redis
RDS Proxy ◆	Connection pooling

3.6 6. Application Integration & Messaging ★

Service	Purpose
SQS ★	Managed message queues
SNS ★	Pub/sub notifications
EventBridge ★	Event bus, scheduling
Step Functions ◆	Workflow orchestration

MQ ♦	Managed ActiveMQ/RabbitMQ
AppSync ♦	GraphQL API
API Gateway ★	REST/WebSocket/HTTP APIs

3.7 7. Developer Tools & CI/CD ★

Service	Purpose
CodeCommit ♦	Git repos (legacy; many use GitHub)
CodeBuild ★	Managed build
CodeDeploy ♦	Deployment automation
CodePipeline ★	CI/CD orchestration
CodeArtifact ♦	Package repository
Cloud9 ♦	Cloud IDE
CDK / SAM / Copilot ♦	IaC & serverless/container tooling
X-Ray ♦	Distributed tracing
CloudWatch Synthetics ♦	Canary monitoring

3.8 8. Observability & Management ★

Service	Purpose
CloudWatch ★	Metrics, alarms, dashboards, logs
CloudWatch Logs Insights ★	Log query
AWS X-Ray ♦	Tracing
Systems Manager ★	Ops hub (Parameter Store, Patch, Session)
CloudTrail ★	Audit
AWS Health Dashboard ★	Service events
Trusted Advisor ♦	Best-practice checks
Personal Health Dashboard ★	Account-specific events
Service Quotas ♦	Limit management
OpsWorks ◇	Legacy Chef/Puppet (avoid for new projects)

3.9 9. Security, Identity & Compliance ★

Service	Purpose
IAM ★	(see section 1)
KMS ★	Encryption key management

Secrets Manager ★	Rotating secrets
Certificate Manager (ACM) ★	TLS certificates
WAF & Shield ◆	Web firewall & DDoS
GuardDuty ★	Threat detection
Inspector ◆	Vulnerability scanning
Macie ◆	S3 data classification
Security Hub ◆	Central security findings
Detective ◇	Security investigation
Firewall Manager ◆	Central WAF/rule management
Artifact ◆	Compliance reports
Audit Manager ◇	Continuous audit evidence
CloudHSM ◇	Dedicated HSM
Cognito ◆	User pools & identity for apps
Verified Permissions ◇	Cedar policy engine
IAM Access Analyzer ◆	External access findings

3.10 10. Migration & Transfer ◆

Service	Purpose
Migration Hub	Migration tracking
Application Migration Service (MGN)	Lift-and-shift
Database Migration Service (DMS)	DB replication
Server Migration Service	(legacy → MGN)
Transfer Family	SFTP/FTPS/FTP to S3
Snow Family	Physical data transfer appliances

3.11 11. Analytics & Big Data ◆

Service	Purpose
Athena ◆	SQL on S3
Glue ◆	ETL & data catalog
EMR ◆	Hadoop/Spark
Kinesis ◆	Streaming (Data Streams, Firehose, Analytics)
QuickSight ◆	BI dashboards
Lake Formation ◆	Data lake governance
MSK ◆	Managed Kafka
OpenSearch Service ◆	Search & analytics (Elasticsearch fork)

Data Pipeline ◇	Legacy orchestration
Clean Rooms ◇	Privacy-preserving analytics

3.12 12. Machine Learning & AI ◇

Service	Purpose
SageMaker ◇	End-to-end ML platform
Bedrock ◇	Foundation models API
Comprehend / Rekognition / Textract / Polly / Transcribe ◇	AI APIs
Forecast / Personalize / Kendra ◇	Specialized ML
CodeWhisperer / Q Developer ◆	AI-assisted development

3.13 13. IoT, Media & Specialized ◇

Service	Purpose
IoT Core / Greengrass	Device connectivity
MediaConvert / Live / Package	Video processing
GameLift	Game server hosting
WorkSpaces / AppStream	Virtual desktops
Connect	Contact center

3.14 14. Cost & Billing ★

Service / Tool	Purpose
Cost Explorer ★	Spend analysis
Budgets ★	Alerts & limits
Cost & Usage Report (CUR) ◆	Detailed billing export
Pricing Calculator ★	Estimate costs
Savings Plans / Reserved Instances ◆	Commitment discounts
Compute Optimizer ◆	Right-sizing recommendations
Billing Conductor ◇	Reseller/MSP billing

3.14.1 Concepts

- Pay-as-you-go vs reserved vs spot
- Tagging strategy for cost allocation
- FinOps culture & unit economics

3.15 15. Well-Architected Framework (conceptual ★)

Six pillars — learn alongside every service:

1. **Operational Excellence** — run & monitor systems
2. **Security** — protect data & assets
3. **Reliability** — recover from failure
4. **Performance Efficiency** — use resources well
5. **Cost Optimization** — avoid unnecessary spend
6. **Sustainability** — minimize environmental impact

Tool: AWS Well-Architected Tool (workload reviews)

3.16 Certification path (recommended order)

Order	Certification	Focus
1	Cloud Practitioner (CLF)	Broad AWS vocabulary
2	Solutions Architect Associate (SAA)	Design & core services
3	Developer Associate (DVA)	CI/CD, serverless, APIs
4	SysOps Administrator (SOA)	Operations & monitoring
5	DevOps Engineer Professional (DOP)	Advanced CI/CD & automation
6	Solutions Architect Professional (SAP)	Complex multi-account design
7	Security Specialty	Deep security services
8	Networking Specialty	VPC, hybrid, TGW

3.17 Hands-on lab progression

1. Create IAM user, MFA, CLI profile
2. VPC with public/private subnets + NAT
3. EC2 + ALB + Auto Scaling
4. S3 static site + CloudFront + Route 53
5. RDS Multi-AZ + Secrets Manager
6. Lambda + API Gateway + DynamoDB
7. ECS Fargate or EKS cluster with CI/CD
8. CloudWatch dashboards + alarms + X-Ray
9. Terraform modules for reusable infra
10. Multi-account landing zone (Organizations + Control Tower)

4 DevOps Handbook — Master Table of Contents

Target length: ~1,200 pages (~480,000 words)

Audience: Beginner → Advanced

Format: Each numbered section ≈ one “page” of ~400 words in the final edition

4.1 How to read this handbook

Work through **Parts I-XII** in order. Labs (marked) add hands-on time beyond page counts. Cross-reference the [AWS Handbook](#) for cloud-specific depth.

4.2 Part I — Introduction & Mindset (Pages 1-60)

Ch	Title	Pages
1	What Is DevOps? History and Culture	1-15
2	CALMS, Three Ways, and Team Topologies	16-30
3	DevOps vs SRE vs Platform Engineering	31-42
4	Career Paths, Certifications, and Learning Strategy	43-60

Written: Chapters 1-3, 5-6, 9, 20-21, 26, 37 (11 chapters + full TOC)

4.3 Part II — Linux & Shell Mastery (Pages 61-140)

Ch	Title	Pages
5	Linux Fundamentals: Filesystem, Users, Permissions	61-85
6	Processes, systemd, and Service Management	86-105
7	Networking on Linux: ip, ss, curl, dig	106-120
8	Bash Scripting for Automation	121-140

Written: [Chapter 5](#), [Chapter 6](#)

4.4 Part III — Git & Collaboration (Pages 141-200)

Ch	Title	Pages
9	Git Internals: Objects, Refs, and the DAG	141-158
10	Branching Strategies and Code Review	159-175
11	GitHub/GitLab Workflows: PRs, Issues, Actions intro	176-200

4.5 Part IV — Networking for DevOps (Pages 201-280)

Ch	Title	Pages
12	OSI Model, TCP/IP, DNS Deep Dive	201-225
13	HTTP/HTTPS, TLS, and Certificates	226-245
14	Load Balancing, Proxies, and CDN Concepts	246-265
15	Cloud Networking Patterns (VPC overview)	266-280

4.6 Part V — Scripting & Programming (Pages 281-360)

Ch	Title	Pages
16	Python for DevOps: boto3, requests, pathlib	281-305
17	Go for CLI Tools and Operators	306-325
18	JSON/YAML, Jinja2, and Templating	326-345
19	Testing Automation Scripts	346-360

4.7 Part VI — Infrastructure as Code (Pages 361-480)

Ch	Title	Pages
20	IaC Principles: State, Idempotency, Drift	361-380
21	Terraform: HCL, Providers, Modules	381-410
22	Terraform: Workspaces, Backends, CI Integration	411-430
23	Ansible: Playbooks, Roles, Inventories	431-450
24	Packer & Golden Images	451-465
25	Policy as Code: OPA, Checkov, Sentinel	466-480

4.8 Part VII — Containers (Pages 481-580)

Ch	Title	Pages
26	Docker: Images, Dockerfile, Multi-stage Builds	481-505
27	Container Networking, Volumes, and Security	506-525
28	Docker Compose and Local Dev Environments	526-540
29	Container Registries and Image Signing	541-560
30	Rootless Containers and Alternatives (Podman)	561-580

4.9 Part VIII — Kubernetes (Pages 581-720)

Ch	Title	Pages
31	K8s Architecture: Control Plane & Node Components	581-605
32	Workloads: Pod, Deployment, StatefulSet, DaemonSet	606-630
33	Services, Ingress, and Network Policies	631-655
34	ConfigMaps, Secrets, and Storage (PV/PVC)	656-675
35	Helm, Kustomize, and GitOps (Argo CD)	676-700
36	RBAC, Pod Security, and Hardening	701-720

4.10 Part IX — CI/CD (Pages 721-840)

Ch	Title	Pages
37	CI/CD Concepts and Pipeline Design	721-745
38	GitHub Actions: Workflows, Secrets, Runners	746-770
39	Jenkins: Pipelines as Code (Declarative)	771-790
40	GitLab CI and Multi-platform Comparison	791-810
41	Deployment Strategies: Blue/Green, Canary, Feature Flags	811-825
42	DORA Metrics and Continuous Improvement	826-840

4.11 Part X — Observability & SRE (Pages 841-960)

Ch	Title	Pages
43	Monitoring Fundamentals: Metrics, Logs, Traces	841-865
44	Prometheus & Grafana Stack	866-890
45	ELK/EFK and Log Aggregation	891-910
46	OpenTelemetry and Distributed Tracing	911-930
47	SLOs, Error Budgets, and Incident Response	931-945
48	Chaos Engineering and Game Days	946-960

4.12 Part XI — Security (DevSecOps) (Pages 961-1080)

Ch	Title	Pages
49	Shift-Left Security in the Pipeline	961-980
50	SAST, DAST, SCA, and Container Scanning	981-1000
51	Secrets Management: Vault, SOPS, External Secrets	1001-1020

52	Supply Chain: SBOM, Sigstore, SLSA	1021-1040
53	Runtime Security: Falco, OPA Gatekeeper	1041-1060
54	Compliance Automation and Audit Trails	1061-1080

4.13 Part XII — Advanced Topics (Pages 1081-1200)

Ch	Title	Pages
55	Platform Engineering & Internal Developer Platforms	1081-1100
56	Multi-Cloud and Hybrid Patterns	1101-1115
57	FinOps and Cost-Aware Engineering	1116-1130
58	MLOps Overview for DevOps Engineers	1131-1145
59	Capstone: End-to-End Production System Design	1146-1180
60	Appendix: Command Cheatsheets, Glossary, Index	1181-1200

4.14 Expansion status

Part	Status
I	Ch 1-3, 5-6, 9, 20-21, 26, 37 written (~120 pages of content)

To request a chapter: *“Expand DevOps Handbook Chapter 21 (Terraform modules)”*

5 Chapter 1: What Is DevOps? History and Culture

DevOps Handbook — Part I, Pages 1–15

5.1 1.1 The problem DevOps solves

Before DevOps, software organizations often split into two camps:

- **Development** — wrote features quickly, measured by release velocity.
- **Operations** — kept systems stable, measured by uptime and incident count.

These goals frequently conflicted. Developers threw code “over the wall”; operators resisted change because every deployment risked an outage. Releases were rare, painful, and manual. When something broke in production, blame flowed in both directions.

DevOps is not a single tool or job title. It is a **cultural and technical movement** that unifies development and operations around shared goals: deliver value to users **fast** and **reliably**.

5.2 1.2 A working definition

***DevOps** is the practice of aligning people, processes, and technology so that software can be built, tested, released, and operated continuously—with feedback loops that shorten the time from idea to production.*

Key outcomes:

Outcome	What it means
Shorter lead time	Hours or days from commit to production, not months
Higher deployment frequency	Many small releases instead of rare big-bang releases
Lower change failure rate	Most changes succeed without rollback
Faster recovery	When failures happen, restore service quickly

These four metrics are known as **DORA metrics** (DevOps Research and Assessment) and are the standard way to measure DevOps maturity.

5.3 1.3 Brief history

Year	Milestone
2007–2009	Flickr, Etsy, and others share “10+ deploys per day” stories
2009	First DevOpsDays conference in Ghent, Belgium (Patrick Debois)
2010	“The Phoenix Project” novel popularizes DevOps narrative
2013	“The DevOps Handbook” (Gene Kim et al.) codifies practices
2016+	Cloud-native, containers, and Kubernetes accelerate adoption
2020s	Platform engineering, GitOps, and AI-assisted ops emerge

The term **DevOps** blends **Development** + **Operations**. It intentionally breaks down silos rather than creating a third silo called “DevOps team” that sits between dev and ops.

5.4 1.4 Culture before tools

A common mistake is buying Jenkins, Kubernetes, or Terraform and calling it “DevOps.” Tools enable DevOps; they do not create it.

Cultural pillars:

1. **Shared ownership** — The team that builds a service also runs it (“you build it, you run it”).
 2. **Blameless postmortems** — Focus on systems and processes, not individuals, after incidents.
 3. **Automation of toil** — Repetitive manual work is eliminated or codified.
 4. **Continuous learning** — Experimentation, feedback, and improvement are expected.
 5. **Customer focus** — Every decision ties back to user value.
-

5.5 1.5 DevOps practices (overview)

You will study each in depth later in this handbook:

Practice	Summary
Version control	Everything in Git: code, config, IaC, docs
CI/CD	Automated build, test, deploy pipelines
Infrastructure as Code	Reproducible environments (Terraform, Ansible)
Monitoring & observability	Know what production is doing in real time
Microservices & containers	Smaller deployable units (optional but common)
Security integration (DevSecOps)	Security checks in the pipeline, not at the end

5.6 1.6 Organizational models

5.6.1 Functional silos (anti-pattern for DevOps)

Separate dev, QA, ops, and security teams with handoffs.

5.6.2 Cross-functional product teams (recommended)

One team owns a product or service end-to-end: code, infra, on-call, roadmap.

5.6.3 DevOps enablement / platform team

A **platform team** builds internal tools (CI templates, K8s clusters, golden paths) so product teams move faster. This is **not** a ticket-taking ops team—it is a **product for developers**.

5.7 1.7 What DevOps is NOT

- **Not “NoOps”** — Operations expertise is more important at scale, not less.
- **Not only for startups** — Enterprises adopt DevOps for compliance and speed.

- **Not replacing developers with operators** — It merges responsibilities skillfully.
- **Not a certification checkbox** — Culture and practice matter more than badges.

5.8 1.8 Your learning path in this handbook

```
graph LR; A[Foundations] --> B[Automation]; B --> C[Containers]; C --> D[CI/CD]; D --> E[Observability]; E --> F[Security]; F --> G[Advanced]; L["Linux, Git, Networking"] --> A; G --> R["Platform & SRE"]
```

Invest 70% of study time in **hands-on labs**. Reading without doing builds false confidence.

5.9 1.9 Chapter summary

- DevOps unifies dev and ops around **speed** and **reliability**.
- **Culture** (shared ownership, blameless learning) precedes tools.
- Measure progress with **DORA metrics**.
- Cross-functional teams and platform engineering are modern expressions of DevOps.

5.10 Lab 1.1 — Reflect on your current workflow

1. Map your (or a sample) team's path from code commit to production.
2. Count manual steps and handoffs.
3. Identify one bottleneck and one automation opportunity.
4. Write a one-paragraph "DevOps vision" for that team.

5.11 Review questions

1. Name the four DORA metrics.
2. Why is “DevOps team” as a silo problematic?
3. What year and event is credited with coining “DevOps”?
4. Give one cultural and one technical practice of DevOps.

Next: Chapter 2 — CALMS, Three Ways, and Team Topologies

6 Chapter 2: CALMS, Three Ways, and Team Topologies

DevOps Handbook — Part I, Pages 16–30

6.1 2.1 The CALMS framework

CALMS is a mnemonic for assessing DevOps maturity across five dimensions:

Letter	Dimension	Description
C	Culture	Collaboration, trust, shared goals
A	Automation	Eliminate manual, error-prone work
L	Lean	Reduce waste, small batches, flow
M	Measurement	Data-driven decisions, DORA metrics
S	Sharing	Knowledge, tools, failures, successes

6.1.1 Culture (C)

Healthy DevOps culture means developers understand production constraints and operators understand business priorities. Psychological safety allows people to report problems early.

6.1.2 Automation (A)

Automate builds, tests, deployments, infrastructure provisioning, and compliance checks. Manual runbooks become code. **Toil**—repetitive operational work that scales linearly with service growth—should be targeted for elimination.

6.1.3 Lean (L)

Inspired by Toyota Production System: optimize **flow**, limit work in progress (WIP), and deliver **small batches**. Large releases hide defects; small releases surface them quickly.

6.1.4 Measurement (M)

You cannot improve what you do not measure. Track deployment frequency, lead time, MTTR, change failure rate, plus business KPIs (revenue, user satisfaction).

6.1.5 Sharing (S)

Internal conferences, shared Slack channels, communal dashboards, and open postmortems spread expertise. Tools and patterns are reused, not reinvented per team.

6.2 2.2 The Three Ways (Gene Kim)

From *The Phoenix Project* and *The DevOps Handbook*, the **Three Ways** describe the principles of DevOps flow:

6.2.1 First Way: Flow (left to right)

Accelerate delivery from **development** → **operations** → **customer**.

- Make work visible (Kanban boards, pipeline status).
- Reduce batch sizes.
- Build quality in (automated tests at every stage).
- Never pass defects downstream.

Example: A commit triggers CI; only green builds reach staging; only approved artifacts reach production.

6.2.2 Second Way: Feedback (right to left)

Amplify feedback from **production** → **development**.

- Monitor user experience and system health.
- Swarm on problems; shorten feedback loops.
- Developers get production access (with guardrails).

Example: Error rate spike in Grafana pages the on-call engineer; linked traces show the bad deploy; rollback is automated.

6.2.3 Third Way: Continuous learning and experimentation

Create a culture of **improvement** and **risk-taking**.

- Blameless postmortems.
- Reserve time for improvement (20% ops time, hack days).
- Rehearse failures (game days, chaos engineering).

6.3 2.3 Team Topologies (Skelton & Pais)

Modern DevOps organizations structure teams intentionally:

Team type	Purpose
Stream-aligned	Delivers product features; primary value stream
Platform	Provides self-service infra (K8s, CI, databases)
Enabling	Coaches stream teams on new tech (temporary)
Complicated-subsystem	Owns deep specialty (e.g., video encoding, ML)

6.3.1 Interaction modes

- **Collaboration** — Two teams work closely for discovery.
- **X-as-a-Service** — Platform provides API/portal; stream teams consume.
- **Facilitating** — Enabling team mentors others.

Anti-pattern: One ops team serves 50 dev teams via tickets.

Better: Platform team offers paved roads; stream teams deploy autonomously.

6.4 2.4 Conway's Law and reverse Conway maneuver

“Organizations design systems that mirror their communication structure.” — Melvin Conway

If dev and ops are separate departments, you will get separate deployment artifacts and fragile handoffs.

Reverse Conway maneuver: Structure teams the way you want the architecture to look. Want autonomous microservices? Give teams end-to-end ownership of each service.

6.5 2.5 Maturity model (informal)

Level	Characteristics
1 — Initial	Manual deploys, silos, hero culture
2 — Managed	Some CI, version control, basic monitoring
3 — Defined	IaC, automated testing, standardized pipelines
4 — Measured	SLOs, DORA tracking, blameless postmortems
5 — Optimizing	Continuous experimentation, platform self-service

Use this for self-assessment, not as a rigid certification ladder.

6.6 2.6 Chapter summary

- **CALMS** evaluates Culture, Automation, Lean, Measurement, Sharing.
 - The **Three Ways** optimize flow, feedback, and learning.
 - **Team Topologies** align org structure with delivery goals.
 - Apply **Conway's Law** consciously when designing teams.
-

6.7 Lab 2.1 — CALMS assessment

Score your team 1–5 on each CALMS letter. Pick the lowest score and draft one actionable improvement per dimension.

6.8 Review questions

1. What does the “L” in CALMS stand for, and what Lean concept supports it?
 2. Describe the First Way in one sentence.
 3. Name the four team types in Team Topologies.
 4. What is a reverse Conway maneuver?
-

Next: Part II — [Chapter 5: Linux Fundamentals](#)

7 Chapter 3: DevOps vs SRE vs Platform Engineering

DevOps Handbook — Part I, Pages 31–42

7.1 3.1 Three related disciplines

DevOps, **Site Reliability Engineering (SRE)**, and **Platform Engineering** are often confused because they share goals: reliable systems, fast delivery, and reduced toil. They are complementary, not competing.

Discipline	Origin	Primary focus
DevOps	Movement (2009+)	Culture + practices bridging dev and ops
SRE	Google (2003+, public ~2016)	Engineering approach to operations at scale
Platform Engineering	Industry evolution (2020s)	Internal platforms that accelerate product teams

7.2 3.2 Site Reliability Engineering (SRE)

Google introduced **SRE** as “what happens when you ask a software engineer to design an operations function.”

7.2.1 Core SRE principles

1. **Embrace risk** — 100% uptime is impossible and expensive; use **error budgets** to balance reliability and velocity.
2. **Service level objectives (SLOs)** — Define measurable targets (e.g., 99.9% availability).
3. **Eliminate toil** — Manual, repetitive work that scales with traffic must be automated.
4. **Monitoring** — Systems must be observable; alerts fire on symptoms users feel.
5. **Release engineering** — Safe, gradual rollouts with fast rollback.
6. **Simplicity** — Prefer boring, well-understood technology.

7.2.2 SRE team responsibilities

- Define and track SLOs/SLIs
- On-call rotation and incident response
- Capacity planning
- Performance and reliability testing
- Building automation to replace toil

7.2.3 Error budget example

If SLO = 99.9% monthly availability, you have ~43 minutes of downtime budget. When budget is exhausted, **freeze feature releases** and focus on reliability until budget recovers.

7.3 3.3 Platform Engineering

Platform engineering treats the internal developer platform as a **product** whose customers are other

engineers.

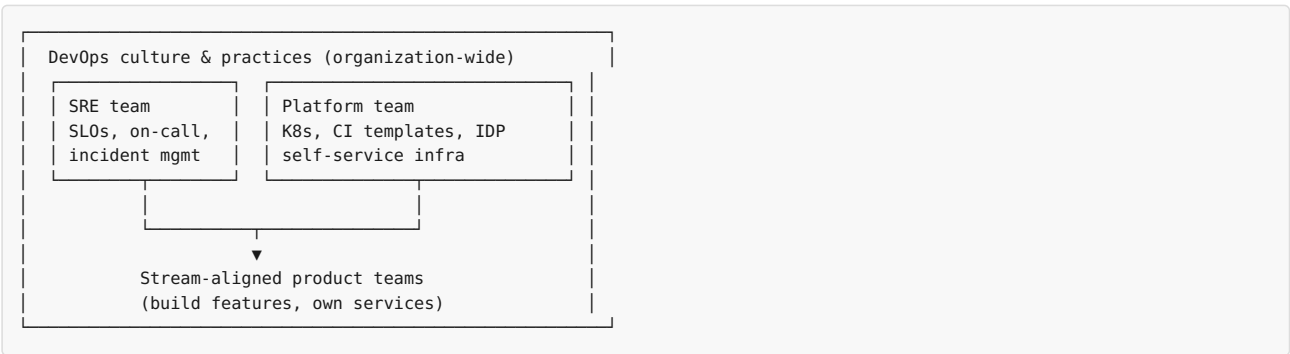
7.3.1 What platform teams build

- Kubernetes clusters with guardrails
- CI/CD templates and golden pipelines
- Self-service databases, caches, and queues
- Developer portals (Backstage) with docs and service catalogs
- Standardized observability (dashboards, tracing, logging)

7.3.2 Platform vs traditional ops

Traditional ops	Platform engineering
Ticket queue for infra requests	Self-service APIs and portals
Snowflake servers	Standardized golden paths
“Call us to deploy”	Teams deploy autonomously with paved roads

7.4 3.4 How they fit together



- **DevOps** sets the cultural north star.
- **SRE** ensures reliability is measurable and engineered.
- **Platform** removes friction so product teams ship faster.

7.5 3.5 Job titles in the market

Titles vary widely. Focus on **responsibilities**, not labels:

Title	Often means
DevOps Engineer	CI/CD, IaC, cloud infra, automation
SRE	SLOs, on-call, reliability tooling, toil reduction
Platform Engineer	Internal platform, K8s, developer experience
Cloud Engineer	AWS/Azure/GCP architecture and implementation
Infrastructure Engineer	Servers, networking, IaC

Many roles blend all three disciplines.

7.6 3.6 Chapter summary

- **SRE** engineers reliability with SLOs, error budgets, and toil automation.
 - **Platform engineering** provides self-service paved roads for developers.
 - **DevOps** is the umbrella culture; SRE and platform are specialized implementations.
-

7.7 Lab 3.1

Draft SLOs for a sample API: pick one SLI (latency or availability), set a target, and calculate the monthly error budget in minutes.

Next: [Chapter 6 — Processes and systemd](#)

8 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

DevOps Handbook — Part II, Pages 61–85

8.1 5.1 Why Linux dominates DevOps

Most servers, containers, and cloud instances run **Linux**. As a DevOps engineer you will SSH into machines, debug production issues, write shell scripts, and configure services daily. Windows matters too (Active Directory, .NET shops), but Linux fluency is non-negotiable.

Common distributions:

Distro	Typical use
Ubuntu	Cloud default, tutorials, desktops
Debian	Stable base for Ubuntu and containers
RHEL / Rocky / Alma	Enterprise, regulated industries
Amazon Linux	AWS-optimized

8.2 5.2 Filesystem hierarchy (FHS)

Path	Purpose
/	Root of entire tree
/home	User home directories (/home/alice)
/root	root user’s home
/etc	Configuration files
/var	Variable data: logs (/var/log), caches
/tmp	Temporary files (often cleared on reboot)
/usr	User programs and libraries
/bin , /sbin	Essential binaries
/opt	Optional third-party software
/proc , /sys	Virtual filesystems for kernel info

Everything is a **file** in Unix—including devices and sockets.

8.3 5.3 Essential navigation commands

```
pwd           # Print working directory
ls -lah       # List files (human-readable, including hidden)
cd /etc/nginx # Change directory
cd ..         # Parent directory
cd ~          # Home directory
```

```
tree -L 2 /etc      # Tree view (install: apt install tree)
```

```
cp source dest           # Copy
cp -r dir1 dir2          # Copy directory recursively
mv old new               # Move or rename
rm file                  # Delete (careful!)
rm -rf directory          # Delete directory recursively (very careful!)
mkdir -p path/to/dir     # Create nested directories
touch file.txt            # Create empty file or update timestamp
cat file                  # Print file contents
less /var/log/syslog      # Paginated view (q to quit)
head -n 20 file           # First 20 lines
tail -f /var/log/app.log  # Follow log in real time
```

```
find /var/log -name "*.log" -mtime -7 # Logs modified in 7 days
find / -type f -size +100M 2>/dev/null # Files larger than 100MB
locate nginx.conf # Fast index search (updatedb)
```

```
id                # Current user UID, GID, groups
whoami
cat /etc/passwd   # User accounts
cat /etc/group     # Groups
sudo command      # Run as root (if permitted)
su - username      # Switch user
```

Each file has three permission sets for **user (u)**, **group (g)**, **others (o)**:

[illegible]

```
chmod 755 script.sh      # rwXR-XR-X
chmod u+x script.sh      # Add execute for user
chown alice:developers file
chgrp developers file
```

8.5.3 Special permissions

- **setuid** — Run as file owner (e.g., `passwd`)
 - **setgid** — Run as group; dirs inherit group
 - **sticky bit** — On `/tmp` , only owner can delete own files
-

8.6 5.6 Package management (Debian/Ubuntu)

```
sudo apt update           # Refresh package index
sudo apt upgrade -y       # Upgrade installed packages
sudo apt install nginx -y # Install package
apt search nginx          # Search
apt show nginx            # Package info
sudo apt remove nginx     # Remove
dpkg -l | grep nginx      # List installed
```

RHEL/Rocky equivalent: `dnf install` , `dnf update` .

8.7 5.7 Text processing (daily DevOps tools)

```
grep -r "ERROR" /var/log/app/ # Search recursively
grep -E "timeout|refused" log.txt # Extended regex
awk '{print $1}' access.log    # Column extraction
sed 's/old/new/g' file.txt     # Stream editor
sort file | uniq -c | sort -rn | head # Count unique lines
wc -l file                    # Line count
```

Pipes (|) chain commands: stdout of one → stdin of next.

8.8 5.8 Environment and shell

```
echo $PATH                # Executable search path
export MY_VAR=hello       # Set environment variable
env                       # All environment variables
which python3             # Path to executable
alias ll='ls -lah'        # Command shortcut
```

Shell config files: `~/.bashrc` , `~/.profile` (sourced on login).

8.9 5.9 SSH — remote access

```
ssh user@hostname
ssh -i ~/.ssh/mykey.pem ec2-user@1.2.3.4
scp file user@host:/remote/path
rsync -avz ./local/ user@host:/remote/ # Efficient sync
```

`~/.ssh/config` simplifies connections:

```
Host prod-web
  HostName 10.0.1.50
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
```

Then: `ssh prod-web`

8.10 5.10 Chapter summary

- Linux FHS organizes `/etc`, `/var`, `/home`, and more.
 - Master `ls`, `cd`, `cp`, `mv`, `rm`, `find`, `grep`, `tail -f`.
 - Permissions: `user/group/other` + `chmod` / `chown`.
 - Use `apt` / `dnf` for packages; SSH for remote ops.
-

8.11 Lab 5.1 — Linux sandbox

1. Create user `devops-lab` with home directory.
 2. Create `~/scripts/deploy.sh` with `#!/bin/bash` and `echo "Deploying..."`.
 3. Set permissions to `755`; run it.
 4. Write a one-liner to count ERROR lines in a sample log file.
 5. Configure SSH key auth to a local VM or cloud instance.
-

8.12 Review questions

1. What directory holds system configuration files?
 2. What does permission `644` mean in symbolic notation?
 3. How do you follow a log file in real time?
 4. What is the difference between `cp` and `mv`?
-

Continue: Chapter 6 — Processes, systemd, and Service Management (outline in TOC)

9 Chapter 6: Processes, systemd, and Service Management

DevOps Handbook — Part II, Pages 86–105

9.1 6.1 Understanding Linux processes

A **process** is a running instance of a program. Each has a **PID** (Process ID), owner, state, and resource usage.

```
ps aux          # All processes (BSD style)
ps -ef         # All processes (POSIX style)
top            # Interactive process viewer
htop          # Enhanced top (install separately)
pgrep nginx    # Find PIDs by name
pkill -HUP nginx # Send signal to processes by name
```

9.1.1 Process states

State	Code	Meaning
Running	R	Executing or runnable
Sleeping	S	Waiting for event
Zombie	Z	Terminated but not reaped by parent
Stopped	T	Paused (e.g., Ctrl+Z)

9.1.2 Signals

```
kill -15 <pid> # SIGTERM – graceful shutdown (default)
kill -9 <pid>  # SIGKILL – force kill (last resort)
kill -HUP <pid> # SIGHUP – reload config (many daemons)
```

9.2 6.2 systemd — modern service manager

Most Linux distros use **systemd** to boot the system and manage services (units).

```
systemctl status nginx
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
systemctl reload nginx      # Reload config without full restart
systemctl enable nginx     # Start on boot
systemctl disable nginx
systemctl list-units --type=service --state=running
journalctl -u nginx -f     # Follow service logs
journalctl -u nginx --since "1 hour ago"
```

9.2.1 Unit files

Service definitions live in `/etc/systemd/system/` or `/lib/systemd/system/`.

Example `/etc/systemd/system/myapp.service`:

```
[Unit]
```

```

Description=My Application
After=network.target

[Service]
Type=simple
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
Restart=on-failure
RestartSec=5
Environment=PORT=8080

[Install]
WantedBy=multi-user.target

```

After creating or editing a unit:

```

sudo systemctl daemon-reload
sudo systemctl enable --now myapp

```

9.3 6.3 Cron and timers

9.3.1 cron — scheduled tasks

```

crontab -e      # Edit user crontab
crontab -l      # List crontab

```

Cron format: minute hour day month weekday command

```

# Every day at 2:30 AM – backup database
30 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1

```

System-wide jobs: /etc/cron.d/, /etc/cron.daily/, etc.

9.3.2 systemd timers (preferred on modern systems)

Timers are more flexible and integrate with journald logging.

```

# /etc/systemd/system/backup.timer
[Unit]
Description=Daily backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target

```

```

systemctl enable --now backup.timer
systemctl list-timers

```

9.4 6.4 Resource limits and troubleshooting

```

# CPU and memory per process
ps -o pid,user,%cpu,%mem,cmd -p <pid>

# Open files by process
lsof -p <pid>
ss -tlnp      # Listening TCP ports with process

```

```
df -h          # Disk usage
free -h        # Memory usage
dmesg | tail    # Kernel messages
```

9.4.1 Common production issues

Symptom	Likely cause	Check
Service won't start	Bad config, missing binary	<code>journalctl -u service -n 50</code>
Port in use	Another process bound	<code>ss -tlnp \ grep :80</code>
Disk full	Logs, temp files	<code>df -h, du -sh /var/log/*</code>
OOM killed	Memory exhaustion	<code>dmesg \ grep -i oom</code>

9.5 6.5 Chapter summary

- Manage processes with `ps`, `kill`, and signals.
- Use **systemctl** and **journalctl** for services on systemd systems.
- Schedule jobs with cron or systemd timers.
- Check logs first when debugging service failures.

9.6 Lab 6.1

1. Create a simple bash script that writes a timestamp to a log every minute.
2. Run it as a systemd service + timer.
3. Practice `systemctl status`, `journalctl -f`, and graceful stop/start.

Next: [Chapter 9 — Git Internals](#)

10 Chapter 9: Git Internals — Objects, Refs, and the DAG

DevOps Handbook — Part III, Pages 141–158

10.1 9.1 Git is a content-addressable filesystem

At its core, Git stores **snapshots** of your project, not just diffs. Every piece of content is identified by a **SHA-1 hash** (Git is migrating to SHA-256).

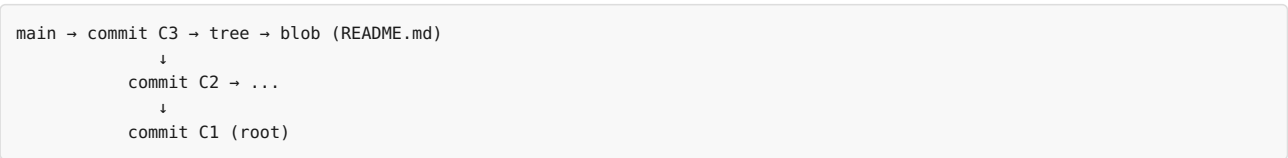
Four object types:

Type	Contains
blob	File content
tree	Directory listing (names → blobs/trees)
commit	Snapshot + metadata (author, message, parent)
tag	Named pointer to a commit (annotated tags)

10.2 9.2 How a commit is stored

When you `git add` and `git commit`:

1. Git compresses file content into **blobs**.
2. Directory structure becomes **trees**.
3. A **commit** object points to the root tree and parent commit(s).
4. Branch name (e.g., `main`) is a **ref** — a pointer to the latest commit.



This forms a **DAG** (directed acyclic graph). Merge commits have two parents.

10.3 9.3 Essential Git commands (beyond basics)

```
git log --oneline --graph --all
git show <commit>           # Commit details + diff
git diff HEAD~1             # Compare with previous commit
git stash push -m "wip"
git stash pop
git cherry-pick <commit>
git rebase main              # Replay commits on top of main
git reflog                  # History of HEAD movements (recovery!)
```

10.3.1 Undoing mistakes

```
git restore file.txt        # Discard working tree changes
```

```
git restore --staged file.txt # Unstage
git reset --soft HEAD~1      # Undo commit, keep changes staged
git reset --hard HEAD~1      # Undo commit, discard changes (dangerous)
git revert <commit>           # New commit that undoes a commit (safe for shared branches)
```

10.4 9.4 Branches and merging

```
git branch feature/login
git switch feature/login      # or: git checkout feature/login
git merge feature/login      # Merge into current branch
git merge --no-ff feature/login # Always create merge commit
```

10.4.1 Merge vs rebase

Merge	Rebase
Preserves history	Linear history
Creates merge commits	Rewrites commits
Safe for shared branches	Use only on local/feature branches

Golden rule: Never rebase commits that others have pulled.

10.5 9.5 Remote repositories

```
git remote -v
git fetch origin          # Download objects, don't merge
git pull origin main      # fetch + merge
git push origin main
git push -u origin feature/x # Set upstream
```

10.5.1 Tracking branches

`origin/main` is a **remote-tracking branch** — your local copy of what's on the server.

10.6 9.6 .gitignore and hooks

10.6.1 .gitignore

```
node_modules/
*.log
.env
dist/
.terraform/
```

10.6.2 Git hooks (`.git/hooks/`)

Scripts that run on events: `pre-commit`, `commit-msg`, `pre-push`. Used for linting, tests, and commit message format.

Example with **pre-commit** framework:

```
pip install pre-commit
# .pre-commit-config.yaml defines hooks
```

10.7 9.7 Chapter summary

- Git stores blobs, trees, commits, and tags; branches are movable pointers.
 - Use `reflog` to recover “lost” commits.
 - Prefer **merge** on shared branches; **rebase** to clean up local work.
 - Automate quality with hooks and `.gitignore`.
-

10.8 Lab 9.1

1. Create a repo with 5 commits on `main`.
 2. Create a feature branch, make 2 commits, merge with `--no-ff`.
 3. Use `git log --graph --oneline --all` to visualize the DAG.
 4. Reset hard to an earlier commit, then recover using `git reflog`.
-

Next: [Chapter 20 — IaC Principles](#)

11 Chapter 20: IaC Principles — State, Idempotency, and Drift

DevOps Handbook — Part VI, Pages 361–380

11.1 20.1 What is Infrastructure as Code?

Infrastructure as Code (IaC) means defining servers, networks, databases, and cloud resources in **machine-readable files** stored in version control—treated like application code.

Benefits:

- **Reproducibility** — Identical dev/staging/prod environments
- **Auditability** — Git history shows who changed what
- **Speed** — Provision in minutes, not days
- **Documentation** — Code is the source of truth
- **Disaster recovery** — Rebuild from code

11.2 20.2 Declarative vs imperative

Style	You specify	Example tools
Declarative	Desired end state	Terraform, CloudFormation, Kubernetes YAML
Imperative	Steps to execute	Ansible (mostly), shell scripts, AWS CLI scripts

Declarative tools compare desired state to actual state and compute a plan. **Imperative** tools run commands in sequence.

Most modern IaC is declarative with imperative escape hatches.

11.3 20.3 Idempotency

An operation is **idempotent** if running it multiple times produces the same result.

```
# Ansible – idempotent: "ensure nginx is installed and running"
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Start nginx
  service:
    name: nginx
    state: started
```

Running the playbook twice does not install nginx twice or restart unnecessarily.

Why it matters: CI/CD pipelines re-run constantly; non-idempotent scripts cause drift and outages.

11.4 20.4 State management

Tools like **Terraform** track what they created in a **state file** (`terraform.tfstate`).

State maps resource addresses to real cloud IDs:

```
{
  "resources": [{
    "type": "aws_s3_bucket",
    "name": "logs",
    "instances": [{
      "attributes": { "id": "my-app-logs-bucket", "arn": "arn:aws:s3:::..." }
    }]
  }]
}
```

11.4.1 State best practices

1. **Remote state** — Store in S3 + DynamoDB lock (AWS), GCS, Terraform Cloud
2. **Never commit secrets** in state (may contain sensitive values)
3. **Locking** — Prevent concurrent `terraform apply` from corrupting state
4. **State encryption** at rest

11.5 20.5 Configuration drift

Drift occurs when live infrastructure diverges from IaC definitions (manual console changes, emergency fixes).

Detection:

- `terraform plan` shows drift before apply
- AWS Config rules
- Scheduled drift detection jobs

Remediation:

- **Reconcile** — Apply IaC to restore desired state
- **Import** — Bring manual resources under IaC management
- **Prevent** — Restrict console access; require IaC PRs

11.6 20.6 IaC workflow in teams

Developer → PR with .tf changes → CI runs `terraform plan` → Review → Merge
→ CD runs `terraform apply` → Slack notification

11.6.1 Directory structure (example)

```
infrastructure/
├── modules/
│   ├── vpc/
│   ├── eks/
│   └── rds/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── README.md
```

Each environment calls shared modules with different variables.

11.7 20.7 Testing IaC

Level	Tool	What it checks
Lint	<code>terraform fmt</code> , <code>tflint</code> , <code>cfn-lint</code>	Syntax and style
Static security	<code>checkov</code> , <code>tfsec</code> , <code>kics</code>	Misconfigurations
Plan	<code>terraform plan</code> in CI	Expected changes
Integration	Terratest, kitchen- terraform	Real deploy in test account

11.8 20.8 Chapter summary

- IaC makes infrastructure versioned, repeatable, and reviewable.
- Prefer **declarative**, **idempotent** tools.
- Manage **state** remotely with locking; detect and fix **drift**.

11.9 Lab 20.1

1. Manually change a tag on a Terraform-managed resource in the AWS console.
2. Run `terraform plan` and observe drift output.
3. Run `terraform apply` to reconcile.

Next: [Chapter 21 — Terraform](#)

12 Chapter 21: Terraform — HCL, Providers, and Modules

DevOps Handbook — Part VI, Pages 381–410

12.1 21.1 Terraform overview

Terraform (HashiCorp / OpenTofu fork) is the most widely used multi-cloud IaC tool. You write **HCL** (HashiCorp Configuration Language); Terraform calls cloud APIs via **providers**.

```
terraform init      # Download providers, init backend
terraform fmt       # Format code
terraform validate  # Syntax check
terraform plan      # Preview changes
terraform apply     # Create/update/destroy resources
terraform destroy   # Tear down (use carefully)
```

12.2 21.2 Basic configuration

`main.tf`:

```
terraform {
  required_version = ">= 1.5"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket     = "my-terraform-state"
    key        = "prod/vpc/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt    = true
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "logs" {
  bucket = "${var.project_name}-logs-${var.environment}"

  tags = {
    Project      = var.project_name
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}
```

`variables.tf`:

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}
```

```
variable "project_name" {
  type = string
}

variable "environment" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}
```

outputs.tf:

```
output "logs_bucket_name" {
  value      = aws_s3_bucket.logs.id
  description = "Name of the logs S3 bucket"
}
```

Apply with: `terraform apply -var="project_name=myapp" -var="environment=dev"`

12.3 21.3 Resources, data sources, and dependencies

12.3.1 Data sources — read existing infrastructure

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
}
```

12.3.2 Implicit vs explicit dependencies

Terraform infers dependencies from references. Use `depends_on` when ordering matters but no attribute reference exists.

12.4 21.4 Modules — reusable infrastructure

modules/vpc/main.tf:

```
resource "aws_vpc" "this" {
  cidr_block      = var.cidr_block
  enable_dns_hostnames = true
  tags = { Name = var.name }
}

resource "aws_subnet" "public" {
  count            = length(var.public_subnet_cidrs)
  vpc_id          = aws_vpc.this.id
  cidr_block      = var.public_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  map_public_ip_on_launch = true
}
```

Using the module:

```
module "vpc" {
  source = "../modules/vpc"

  name           = "prod-vpc"
  cidr_block     = "10.0.0.0/16"
  public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
  availability_zones = ["us-east-1a", "us-east-1b"]
}
```

Publish modules to **Terraform Registry** or private registries for org-wide reuse.

12.5 21.5 Lifecycle meta-arguments

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = false # Set true for prod databases
    ignore_changes        = [tags["LastReboot"]] # Ignore specific drift
  }
}
```

12.6 21.6 Workspaces and environments

```
terraform workspace list
terraform workspace new staging
terraform workspace select prod
```

Workspaces share the same backend key by default—often **separate directories per environment** (with separate state keys) is clearer for production.

12.7 21.7 CI/CD integration

GitHub Actions example (plan on PR):

```
- name: Terraform Plan
  run: |
    terraform init -input=false
    terraform plan -input=false -no-color -out=tfplan
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

Apply only from protected `main` branch after approval.

12.8 21.8 Chapter summary

- Terraform workflow: **init** → **plan** → **apply** with remote state.
- Use **variables**, **outputs**, and **modules** for reuse.
- Run `terraform plan` in CI on every PR; apply from trusted pipelines only.

12.9 Lab 21.1

1. Write Terraform to create an S3 bucket with versioning and encryption.
2. Extract a VPC module with 2 public subnets.
3. Configure S3 backend for remote state.
4. Run plan/apply in a free-tier AWS account.

Next: [Chapter 26 — Docker](#)

13 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds

DevOps Handbook — Part VII, Pages 481–505

13.1 26.1 Containers in one paragraph

A **container** packages an application with its dependencies into an isolated process on a shared Linux kernel. Unlike VMs, containers share the host OS kernel—lightweight and fast to start.

Docker is the dominant tooling for building, shipping, and running containers locally and in CI.

13.2 26.2 Core concepts

Concept	Description
Image	Read-only template (layers)
Container	Running instance of an image
Dockerfile	Recipe to build an image
Registry	Storage for images (Docker Hub, ECR)
Volume	Persistent data outside container lifecycle
Network	Connect containers to each other and the host

13.3 26.3 Essential commands

```
docker pull nginx:1.25-alpine
docker images
docker run -d -p 8080:80 --name web nginx:1.25-alpine
docker ps
docker logs web
docker exec -it web sh
docker stop web && docker rm web
docker build -t myapp:1.0 .
docker tag myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```

13.4 26.4 Dockerfile best practices

```
# Use specific version tags, not :latest in production
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false

COPY . .
RUN npm run build
```

```
# Multi-stage: smaller final image
FROM node:20-alpine AS runtime

RUN addgroup -g 1001 -S app && adduser -u 1001 -S app -G app
WORKDIR /app
USER app

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package.json .

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1

CMD ["node", "dist/server.js"]
```

13.4.1 Best practices checklist

- Use **multi-stage builds** to shrink images
- Run as **non-root** user
- Minimize layers; combine RUN commands where sensible
- Use `.dockerignore` (exclude `node_modules`, `.git`, etc.)
- Pin base image digests for supply chain security
- Add **HEALTHCHECK** for orchestrators

13.5 26.5 Docker Compose for local development

`docker-compose.yml` :

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/app
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d app"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
```

```
docker compose up -d
docker compose logs -f api
docker compose down -v # Remove volumes too
```

13.6 26.6 Security basics

- Scan images: `docker scout cves` or **Trivy**
 - Do not store secrets in images or Dockerfile `ENV`
 - Use read-only root filesystem where possible (`--read-only`)
 - Drop capabilities: `--cap-drop=ALL`
-

13.7 26.7 Chapter summary

- Images are built from Dockerfiles; containers are ephemeral runtime instances.
 - **Multi-stage builds** and non-root users are production essentials.
 - **Docker Compose** orchestrates multi-container local stacks.
-

13.8 Lab 26.1

1. Containerize a simple Node or Python web app.
 2. Add multi-stage build and health check.
 3. Run with Compose alongside PostgreSQL.
 4. Scan the image with Trivy.
-

Next: [Chapter 37 — CI/CD Pipeline Design](#)

14 Chapter 37: CI/CD Concepts and Pipeline Design

DevOps Handbook — Part IX, Pages 721–745

14.1 37.1 Continuous Integration (CI)

CI means developers merge code to a shared branch **frequently** (at least daily), and each merge triggers an **automated build and test** pipeline.

Goals:

- Detect integration bugs early
- Keep `main` always deployable
- Reduce merge hell from long-lived branches

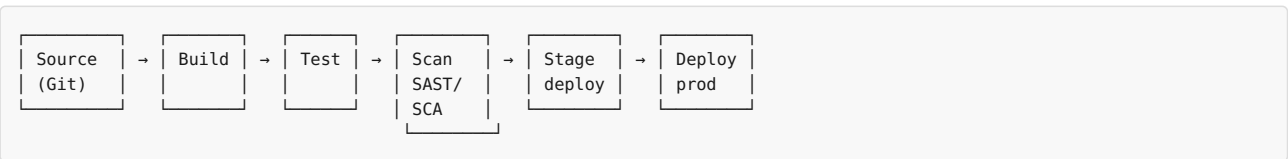
14.2 37.2 Continuous Delivery vs Continuous Deployment

Practice	Definition
Continuous Delivery	Every change is releasable; deploy to prod is a manual (or one-click) decision
Continuous Deployment	Every change that passes tests is automatically deployed to production

Most enterprises use **Continuous Delivery** with approval gates for production.

14.3 37.3 Pipeline stages

A typical pipeline:



Stage	Activities
Source	Webhook on push/PR
Build	Compile, docker build, artifact upload
Test	Unit, integration, contract tests
Scan	SAST, dependency scan, container scan, IaC scan
Stage deploy	Deploy to staging; smoke tests
Prod deploy	Blue/green, canary, or rolling; approval gate

14.4 37.4 GitHub Actions example

.github/workflows/ci.yml:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm

      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage

      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Scan image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: myapp:${{ github.sha }}
          severity: CRITICAL,HIGH
          exit-code: 1

  deploy-staging:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging
        run: ./scripts/deploy.sh staging ${{ github.sha }}
```

14.5 37.5 Branching and trunk-based development

Trunk-based development (recommended for CI/CD):

- Short-lived feature branches (< 2 days)
- Frequent merges to `main`
- Feature flags hide incomplete work

Avoid long-running `develop` branches that diverge from production reality.

14.6 37.6 Deployment strategies

Strategy	How it works	Risk
Rolling	Replace instances gradually	Medium
Blue/green	Switch traffic between two identical envs	Low downtime
Canary	Route small % traffic to new version	Lowest blast radius

Recreate	Tear down old, start new	Downtime
-----------------	--------------------------	----------

14.7 37.7 Pipeline secrets and artifacts

- Store secrets in **GitHub Secrets**, **Vault**, or cloud secret managers—never in YAML.
- Pin action versions (`@v4` , commit SHA) for supply chain security.
- Upload build artifacts (JAR, Docker image digest) for downstream deploy jobs.
- Sign artifacts (Sigstore/cosign) for provenance.

14.8 37.8 DORA metrics in your pipeline

Track:

1. **Deployment frequency** — How often you deploy to prod
2. **Lead time for changes** — Commit to prod duration
3. **Change failure rate** — % of deploys causing incidents
4. **MTTR** — Time to restore after failure

Instrument pipelines to emit these metrics to your observability platform.

14.9 37.9 Chapter summary

- CI validates every merge; CD/CD automates release to staging/prod.
- Design pipelines with **build → test → scan → deploy** stages.
- Prefer trunk-based dev, pinned dependencies, and safe deployment strategies.

14.10 Lab 37.1

Build a full GitHub Actions pipeline for a sample app: lint, test, Docker build, Trivy scan, and deploy to staging on merge to `main`.

Continue: Chapter 38 — GitHub Actions Deep Dive (outline in TOC)

15 AWS Handbook — Master Table of Contents

Target length: ~800 pages (~320,000 words)
Audience: Beginner → Advanced AWS practitioners
Companion: [DevOps Handbook](#)

15.1 Part I — Cloud Foundations (Pages 1-80)

Ch	Title	Pages
1	Introduction to AWS & Shared Responsibility	1-18
2	AWS Global Infrastructure: Regions, AZs, Edge	19-35
3	AWS Account Setup, Billing, and Cost Basics	36-55
4	AWS CLI, CloudShell, and SDK Overview	56-80

Written: Chapters 1-3, 5-6, 9-10, 15, 20, 35 (11 chapters + full TOC)

15.2 Part II — Identity & Access Management (Pages 81-160)

Ch	Title	Pages
5	IAM Users, Groups, Roles, and Policies	81-105
6	IAM Best Practices & Permission Boundaries	106-125
7	IAM Identity Center (SSO) & Federation	126-145
8	AWS Organizations & Service Control Policies	146-160

Written: [Chapter 5](#)

15.3 Part III — Networking (Pages 161-280)

Ch	Title	Pages
9	VPC Fundamentals: CIDR, Subnets, Route Tables	161-185
10	Internet Gateway, NAT, and Routing	186-205
11	Security Groups vs NACLs	206-220
12	Load Balancers: ALB, NLB, GWLB	221-245
13	Route 53: DNS Records and Routing Policies	246-265
14	VPC Peering, Transit Gateway, PrivateLink	266-280

15.4 Part IV — Compute (Pages 281-400)

Ch	Title	Pages
----	-------	-------

15	EC2: Instance Types, AMIs, Launch Templates	281–305
16	Auto Scaling & Elastic Load Balancing Integration	306–325
17	Lambda: Events, Concurrency, and Layers	326–350
18	ECS & Fargate	351–375
19	EKS: Managed Kubernetes on AWS	376–400

15.5 Part V — Storage (Pages 401–480)

Ch	Title	Pages
20	S3: Buckets, Objects, Versioning, Lifecycle	401–425
21	S3 Security, Encryption, and Static Websites	426–445
22	EBS & EFS	446–465
23	AWS Backup & Disaster Recovery Patterns	466–480

15.6 Part VI — Databases (Pages 481–560)

Ch	Title	Pages
24	RDS & Aurora	481–505
25	DynamoDB: Tables, Indexes, Streams	506–525
26	ElastiCache & Caching Patterns	526–540
27	Database Migration (DMS) & DR	541–560

15.7 Part VII — Application Integration (Pages 561–620)

Ch	Title	Pages
28	API Gateway	561–580
29	SQS, SNS, and EventBridge	581–605
30	Step Functions Workflows	606–620

15.8 Part VIII — DevOps on AWS (Pages 621–700)

Ch	Title	Pages
31	CloudFormation Deep Dive	621–645
32	AWS CDK	646–665
33	CodePipeline, CodeBuild, CodeDeploy	666–685
34	Systems Manager & Parameter Store	686–700

15.9 Part IX — Observability & Operations (Pages 701-760)

Ch	Title	Pages
35	CloudWatch Metrics, Alarms, Dashboards	701-720
36	CloudWatch Logs & Insights	721-735
37	X-Ray & Distributed Tracing	736-750
38	CloudTrail & AWS Config	751-760

15.10 Part X — Security & Compliance (Pages 761-800)

Ch	Title	Pages
39	KMS, Secrets Manager, ACM	761-775
40	GuardDuty, Security Hub, WAF, Shield	776-790
41	Well-Architected Review & Capstone Design	791-800

15.11 Expansion status

Part	Status
I-II, III-V, IX	Ch 1-3, 5-6, 9-10, 15, 20, 35 written (~120 pages of content)

To request a chapter: *“Expand AWS Handbook Chapter 15 (EC2 deep dive)”*

16 Chapter 1: Introduction to AWS & the Shared Responsibility Model

AWS Handbook — Part I, Pages 1–18

16.1 1.1 What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is a comprehensive **cloud computing platform** offering on-demand compute, storage, databases, networking, machine learning, and 200+ other services. Instead of buying physical servers, you provision resources in minutes and pay primarily for what you use.

16.1.1 Why AWS matters for DevOps

- **API-driven** — Every service is controllable via CLI, SDK, and IaC.
 - **Global scale** — Deploy in multiple regions for resilience and latency.
 - **Managed services** — Offload undifferentiated heavy lifting (RDS, EKS, Lambda).
 - **Ecosystem** — Largest cloud market share; extensive documentation and community.
-

16.2 1.2 Cloud service models

Model	You manage	Provider manages	AWS examples
IaaS	OS, apps, data	Hardware, network, hypervisor	EC2, VPC, EBS
PaaS	Apps, data	Runtime, OS, infra	Elastic Beanstalk, App Runner
SaaS	Configuration, data	Everything else	WorkSpaces, Chime
FaaS	Function code	Everything else	Lambda

Containers blur lines: ECS/EKS give you orchestration (between IaaS and PaaS).

16.3 1.3 The Shared Responsibility Model

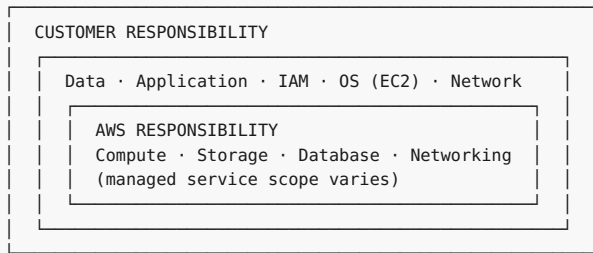
Security and compliance are **shared** between AWS and the customer. The boundary shifts by service type.

16.3.1 AWS is responsible for Security OF the Cloud

- Physical data centers
- Hardware and global network
- Virtualization layer
- Managed service patching (for fully managed services)

16.3.2 You are responsible for Security IN the Cloud

- Data classification and encryption choices
- IAM users, roles, and policies
- Operating system patches (on EC2)
- Network configuration (security groups, NACLs)
- Application code security



16.3.3 Example: RDS vs EC2 MySQL

Task	EC2 + self-managed MySQL	Amazon RDS
Patch MySQL	You	AWS
Patch OS	You	AWS
Configure security groups	You	You
Encrypt data at rest	You (configure)	You (enable)
Physical security	AWS	AWS

16.4 1.4 Core AWS concepts

Concept	Definition
Region	Geographic area (e.g., <code>us-east-1</code> , <code>eu-west-1</code>)
Availability Zone (AZ)	Isolated data center within a region
Account	Billing and security boundary (12-digit ID)
Resource	Any AWS entity (EC2 instance, S3 bucket, etc.)
ARN	Amazon Resource Name — unique identifier
Tag	Key-value metadata on resources

Example ARN:

```
arn:aws:s3:::my-bucket
arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234
```

16.5 1.5 AWS console, CLI, and IaC

Three ways to interact with AWS:

1. **Management Console** — Web UI; great for learning and ad-hoc tasks.
2. **AWS CLI** — Scriptable; essential for automation.
3. **Infrastructure as Code** — CloudFormation, CDK, Terraform for reproducible environments.

DevOps engineers spend most time in **CLI + IaC**, using the console for debugging.

16.6 1.6 Pricing fundamentals

- **On-Demand** — Pay per hour/second; no commitment.
- **Reserved Instances / Savings Plans** — 1–3 year commitment; lower cost.
- **Spot Instances** — Unused capacity; cheap but can be interrupted.
- **Free Tier** — Limited free usage for 12 months (and some always-free services).

Always set billing alarms (AWS Budgets) on new accounts.

16.7 1.7 Well-Architected Framework (preview)

AWS defines six pillars for good architecture (covered throughout this handbook):

1. Operational Excellence
 2. Security
 3. Reliability
 4. Performance Efficiency
 5. Cost Optimization
 6. Sustainability
-

16.8 1.8 Chapter summary

- AWS provides on-demand cloud infrastructure and managed services.
 - **Shared responsibility** splits security between AWS and you—know your side.
 - Learn **regions, accounts, ARNs, and tags** early.
 - Use CLI and IaC for production; console for exploration.
-

16.9 Lab 1.1 — Create your AWS account

1. Sign up at <https://aws.amazon.com> (use a dedicated email).
 2. Enable **MFA** on the root account; do not use root daily.
 3. Create an **IAM admin user** for daily work.
 4. Set a **billing budget** alert at \$10 or your comfort threshold.
 5. Explore the console: EC2, S3, IAM dashboards.
-

16.10 Review questions

1. What is the difference between IaaS and PaaS? Give one AWS example of each.
 2. Who patches the OS on an EC2 instance?
 3. What is an ARN used for?
 4. Name two AWS pricing models.
-

Next: [Chapter 2 — Global Infrastructure](#)

17 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

AWS Handbook — Part I, Pages 19–35

17.1 2.1 Global infrastructure overview

AWS operates a worldwide network of data centers organized into:

Region (e.g., us-east-1)

- Availability Zone A (us-east-1a)
- Availability Zone B (us-east-1b)
- Availability Zone C (us-east-1c)
- ... (typically 3+ AZs per region)

Edge Locations (200+) — CloudFront, Route 53, etc.

17.2 2.2 Regions

A **Region** is a separate geographic area. Each region is **fully isolated** from others for most services.

17.2.1 Choosing a region

Factor	Guidance
Latency	Place resources near users
Compliance	Data residency (GDPR, etc.) may require EU regions
Service availability	New services launch in select regions first
Cost	Pricing varies by region (us-east-1 often cheapest)
Disaster recovery	Use a second region for critical workloads

Naming: us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

17.2.2 Regional vs global services

Regional (must specify region)	Global (single endpoint)
EC2, RDS, VPC, Lambda	IAM, Route 53, CloudFront
S3 (bucket tied to region)	AWS Organizations

17.3 2.3 Availability Zones (AZs)

An **AZ** is one or more discrete data centers with independent power, networking, and cooling within a region. AZs are connected by **low-latency private fiber**.

- Deploy across **multiple AZs** for high availability.
- AZ names like us-east-1a are **account-specific mappings**— 1a in your account may not be the same physical DC as 1a in another account.

17.3.1 Multi-AZ patterns

Pattern	Use case
Active-passive	RDS Multi-AZ failover
Active-active	ALB spanning AZs with EC2 in each
Stretch cluster	EKS node groups per AZ

17.4 2.4 Local Zones, Wavelength, and Outposts

Type	Purpose
Local Zones	Extend region closer to metro areas (ultra-low latency)
Wavelength	5G edge at carrier locations
Outposts	AWS hardware on-premises; hybrid cloud

These are specialized; start with standard regions + AZs.

17.5 2.5 Edge locations and CloudFront

Edge locations cache content close to users via **Amazon CloudFront** (CDN). Route 53 and AWS Shield also use the edge network.

Benefits: - Lower latency for static assets - Reduced origin load - DDoS protection at edge (with Shield)

17.6 2.6 Fault isolation design principles

1. **Never rely on a single AZ** for production.
2. **Design for regional failure** for mission-critical systems (multi-region active-active or pilot light).
3. **Use managed services** that handle AZ redundancy (RDS Multi-AZ, S3 cross-AZ durability).
4. **Test failover** regularly.

17.6.1 S3 durability vs availability

- **S3 Standard** — 99.999999999% (11 nines) durability across AZs in a region.
- **Availability** — Different metric; rare regional outages can still occur.

17.7 2.7 Service quotas (limits)

Each account has **default quotas** per region (e.g., EC2 vCPUs, VPCs per region). Request increases via **Service Quotas** console. Plan capacity before large deployments.

17.8 2.8 Chapter summary

- **Regions** are geographic; **AZs** are isolated DCs within a region.

- Deploy production workloads across **multiple AZs**.
 - **Edge locations** power CloudFront and global services.
 - Know **regional vs global** services when designing architecture.
-

17.9 Lab 2.1 — Explore regions in CLI

```
aws configure                                # Set default region
aws ec2 describe-availability-zones --region us-east-1
aws ec2 describe-regions --output table
```

Document which region you will use for labs and why.

17.10 Review questions

1. How many AZs should a highly available web app use at minimum?
 2. Is IAM a regional or global service?
 3. What AWS service uses edge locations for caching?
 4. Why might you choose `eu-central-1` over `us-east-1`?
-

Next: Part II — [Chapter 5: IAM Fundamentals](#)

18 Chapter 3: AWS Account Setup, Billing, and Cost Basics

AWS Handbook — Part I, Pages 36–55

18.1 3.1 Creating a secure AWS account

18.1.1 Step-by-step

1. **Sign up** at <https://aws.amazon.com> with a dedicated email (not personal if possible for work).
2. **Root account** — Full admin access. Enable **MFA immediately**.
3. **Do not use root** for daily tasks.
4. Create an **IAM admin user** or use **IAM Identity Center** for workforce access.
5. Set up **billing alerts** before launching resources.

18.1.2 Root account checklist

- ☐ MFA enabled (hardware key preferred)
- ☐ Strong unique password in password manager
- ☐ No access keys on root
- ☐ Contact info and alternate contacts set
- ☐ AWS Budgets alert configured

18.2 3.2 AWS Free Tier

Type	Examples
12-month free	750 hrs/month t2/t3.micro EC2, 5 GB S3, RDS limits
Always free	Lambda 1M requests/month, DynamoDB 25 GB
Short trials	Some services have limited trials

Warning: Free tier expires; resources left running can incur charges. Always set billing alarms.

18.3 3.3 Billing and Cost Management

Tool	Purpose
Billing Dashboard	Current month spend
Cost Explorer	Historical analysis, forecasts, filters
AWS Budgets	Email/SNS alerts at thresholds
Cost & Usage Report (CUR)	Granular export to S3 for analysis
Pricing Calculator	Estimate architecture costs before building

18.3.1 Budget example

Create a monthly budget of \$20 with alerts at 50%, 80%, and 100%:

```
aws budgets create-budget \
  --account-id 123456789012 \
  --budget file://budget.json \
  --notifications-with-subscribers file://notifications.json
```

18.4 3.4 Cost optimization fundamentals

1. **Tag everything** — Environment, Project, Owner, CostCenter
2. **Right-size** — Use Compute Optimizer recommendations
3. **Stop dev resources** — Schedule EC2/RDS shutdown nights/weekends
4. **Use Spot** for fault-tolerant batch workloads
5. **S3 lifecycle** — Move old data to IA/Glacier
6. **Reserved capacity** — 1–3 year Savings Plans for steady workloads
7. **Delete unused** — EBS volumes, Elastic IPs, old snapshots

18.5 3.4 Consolidated billing with Organizations

AWS Organizations lets you manage multiple accounts under one payer account:

- **Consolidated billing** — Single invoice, volume discounts
- **SCPs** — Guardrails on what accounts can do
- **Separate accounts** per environment (dev/staging/prod) or per team

Recommended for any serious AWS usage.

18.6 3.5 Support plans (overview)

Plan	Best for
Basic	Free; account and billing support
Developer	Business hours email support
Business	24/7 phone, < 1 hr response for production down
Enterprise	TAM, < 15 min critical response

18.7 3.6 Chapter summary

- Secure the **root account** with MFA; use IAM for daily work.
- Set **budgets and alerts** on day one.
- **Tag** resources and review Cost Explorer weekly during learning.

18.8 Lab 3.1

1. Enable MFA on root.
2. Create IAM admin user `admin-yourname`.
3. Create a \$10 monthly budget with email alert.
4. Enable Cost Explorer (may take 24h for data).

Next: [Chapter 6 — IAM Best Practices](#)

19 Chapter 5: IAM Users, Groups, Roles, and Policies

AWS Handbook — Part II, Pages 81–105

19.1 5.1 What is IAM?

AWS Identity and Access Management (IAM) controls **who** can access **which** AWS resources and **how**. Every API call to AWS is authenticated and authorized through IAM (except public resources like anonymous S3 access).

IAM is **global**—users and roles exist at the account level, not per region.

19.2 5.2 IAM identity types

Identity	Use case
Root user	Account owner; created at signup. Avoid for daily use.
IAM user	Long-term credentials for humans or legacy apps
IAM role	Temporary credentials; assumed by users, services, or other accounts
IAM group	Collection of users with shared permissions

Best practice: Humans use **IAM Identity Center (SSO)** or roles with MFA. Applications and EC2/Lambda use **roles**, not access keys on users.

19.3 5.3 Policies — the core of permissions

A **policy** is a JSON document defining **Allow** or **Deny** for **actions** on **resources** under **conditions**.

19.3.1 Policy structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Read",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-app-bucket",
        "arn:aws:s3::my-app-bucket/*"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

```
}
```

19.3.2 Policy types

Type	Attached to
Identity-based	Users, groups, roles
Resource-based	S3 buckets, SNS topics, etc.
Permission boundaries	Max permissions for identity
SCP	AWS Organizations — account ceiling
Session policies	Limit role session scope

Evaluation logic: Explicit **Deny** always wins. Default is implicit deny until Allow.

19.4 5.4 Users and groups

19.4.1 Create a developer group (example workflow)

1. Create group `Developers`.
2. Attach policy `PowerUserAccess` (or custom least-privilege policy).
3. Create users; add to group.
4. Enable **MFA** for console access.
5. Provide **programmatic access** only if needed (prefer roles).

```
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/PowerUserAccess
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Developers
```

19.5 5.5 Roles and trust policies

A **role** has two policies:

1. **Trust policy** — Who can assume the role
2. **Permission policy** — What the role can do

19.5.1 EC2 instance role example

Trust policy (allows EC2 service to assume):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Attach permission policy allowing `s3:GetObject` on a specific bucket. The EC2 instance receives **temporary credentials** via instance metadata—no hardcoded keys.

19.5.2 Assume role (cross-account or CLI)

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/DeployRole \
  --role-session-name deploy-session
```

19.6 5.6 AWS managed vs customer managed policies

Type	Description
AWS managed	Maintained by AWS (<code>ReadOnlyAccess</code> , <code>AdministratorAccess</code>)
Customer managed	You create and reuse across identities
Inline	Embedded in single user/group/role; harder to reuse

Prefer **customer managed** policies for team standards; avoid `AdministratorAccess` except break-glass accounts.

19.7 5.7 Least privilege in practice

1. Start with **read-only** access; add permissions as needed.
2. Use **IAM Access Analyzer** to find overly broad access.
3. Scope resources with ARNs, not `"Resource": "*" .`
4. Use **conditions** (MFA present, source IP, tag-based).
5. Rotate or eliminate **access keys**; prefer roles.

19.7.1 Tag-based access (ABAC)

```
"Condition": {
  "StringEquals": {
    "aws:PrincipalTag/Department": "${aws:ResourceTag/Department}"
  }
}
```

Users tagged `Department=Engineering` access only resources tagged the same.

19.8 5.8 Common IAM actions for DevOps

Action prefix	Service
<code>ec2:*</code>	EC2
<code>s3:*</code>	S3
<code>iam:PassRole</code>	Allow attaching roles to services
<code>sts:AssumeRole</code>	Switch roles
<code>cloudformation:*</code>	Stack operations

`iam:PassRole` is often required when CI/CD deploys resources that need an instance/task role.

19.9 5.9 IAM Access Analyzer & CloudTrail

- **Access Analyzer** — Identifies resources shared externally.
- **CloudTrail** — Logs every IAM API call for audit.

Enable CloudTrail in all regions for security accounts.

19.10 5.10 Chapter summary

- IAM controls authentication and authorization globally.
 - Use **groups** for humans, **roles** for services and temporary access.
 - Policies are JSON with Effect, Action, Resource, Condition.
 - Apply **least privilege**; avoid root and long-lived access keys.
-

19.11 Lab 5.1 — IAM hands-on

1. Create IAM user `lab-user` with MFA.
 2. Create customer managed policy allowing `s3:ListBucket` and `s3:GetObject` on one bucket.
 3. Create role `EC2-S3-ReadRole` with trust for EC2 and the S3 read policy.
 4. Launch EC2 with the role; verify `aws s3 ls` works without configured keys.
 5. Review effective permissions in **IAM Policy Simulator**.
-

19.12 Review questions

1. What is the difference between an IAM user and an IAM role?
 2. What happens when Allow and Deny conflict?
 3. Why should EC2 instances use roles instead of access keys?
 4. What does `sts:AssumeRole` do?
-

Continue: Chapter 6 — IAM Best Practices (outline in TOC)

20 Chapter 6: IAM Best Practices and Permission Boundaries

AWS Handbook — Part II, Pages 106–125

20.1 6.1 The IAM security baseline

Follow these rules in every AWS account:

1. **Root account** — MFA only; no access keys; break-glass use only
2. **No long-lived access keys** for humans — use SSO + temporary credentials
3. **Least privilege** — Grant minimum permissions required
4. **MFA** — Required for console access and sensitive API calls
5. **Rotate keys** — If keys must exist, rotate every 90 days max
6. **Audit** — CloudTrail enabled in all regions; review Access Analyzer findings

20.2 6.2 Prefer roles over users for applications

Identity	Use for
IAM user + keys	Legacy; avoid for new projects
IAM role	EC2, Lambda, ECS, cross-account, CI/CD
IAM Identity Center	Human workforce access

20.2.1 CI/CD role pattern

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:*"
      }
    }
  }]
}
```

GitHub Actions assumes this role—no static AWS keys in GitHub Secrets.

20.3 6.3 Permission boundaries

A **permissions boundary** sets the **maximum** permissions an identity can have—even if policies grant more.

Use for:

- Delegating IAM admin to developers without full account access

- Sandbox accounts with caps

```
aws iam put-user-permissions-boundary \  
  --user-name developer \  
  --permissions-boundary arn:aws:iam::aws:policy/PowerUserAccess
```

20.4 6.4 Policy writing tips

20.4.1 Start with AWS managed, refine to customer managed

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "DeployToSpecificBucket",  
    "Effect": "Allow",  
    "Action": ["s3:PutObject", "s3:GetObject"],  
    "Resource": "arn:aws:s3::my-app-artifacts-prod/*"  
  }]  
}
```

20.4.2 Use IAM Policy Simulator

Test policies before deploying: AWS Console → IAM → Policy Simulator.

20.4.3 Avoid these anti-patterns

- "Action": "*" on production roles
- "Resource": "*" when ARNs are known
- Attaching AdministratorAccess to CI roles
- Sharing one IAM user across a team

20.5 6.5 Service-linked roles

AWS creates **service-linked roles** for services (e.g., AWSServiceRoleForElasticLoadBalancing). Do not delete these unless AWS docs say it's safe.

20.6 6.6 Cross-account access

Pattern: Account A (workload) trusts Account B (CI/CD) via role assumption.

```
# From CI in account B  
aws sts assume-role \  
  --role-arn arn:aws:iam::WORKLOAD_ACCOUNT:role/DeployRole \  
  --role-session-name github-deploy
```

Use **AWS Organizations** SCPs to prevent privilege escalation across accounts.

20.7 6.7 Chapter summary

- Humans: **SSO + MFA**. Apps: **roles**. CI: **OIDC federation**.
- Write **scoped customer-managed policies**; test with Policy Simulator.
- Use **permission boundaries** and **SCPs** for guardrails.

20.8 Lab 6.1

1. Create a CI/CD role trust policy for GitHub OIDC (or a second AWS account).
2. Attach a least-privilege policy for S3 deploy only.
3. Run Access Analyzer and remediate any `Public` findings.

Next: [Chapter 9 — VPC Fundamentals](#)

21 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables

AWS Handbook — Part III, Pages 161–185

21.1 9.1 What is a VPC?

A **Virtual Private Cloud (VPC)** is your isolated network in AWS. You control IP ranges, subnets, routing, and firewalls.

Every AWS account has a **default VPC** per region (with public subnets). Production workloads use **custom VPCs** with explicit design.

21.2 9.2 CIDR and IP planning

CIDR (Classless Inter-Domain Routing) defines IP ranges:

CIDR	Addresses	Usable (AWS reserves 5)
10.0.0.0/16	65,536	~65,531
10.0.1.0/24	256	~251
10.0.1.0/28	16	~11

21.2.1 Example VPC design

VPC: 10.0.0.0/16

Public subnets (ALB, NAT Gateway):
10.0.1.0/24 – us-east-1a
10.0.2.0/24 – us-east-1b

Private subnets (app servers, ECS, Lambda):
10.0.10.0/24 – us-east-1a
10.0.11.0/24 – us-east-1b

Database subnets (RDS, no internet):
10.0.20.0/24 – us-east-1a
10.0.21.0/24 – us-east-1b

Tip: Plan for growth; changing VPC CIDR later is painful. Use /16 for VPC, /24 for subnets.

21.3 9.3 Subnets

Subnets live in **one Availability Zone**. Types:

Type	Route to IGW	Typical workloads
Public	Yes (via route table)	ALB, NAT Gateway, bastion
Private	No direct IGW	App servers, workers
Isolated	No NAT either	Databases

`map_public_ip_on_launch = true` assigns public IPs to instances in public subnets.

21.4 9.4 Route tables

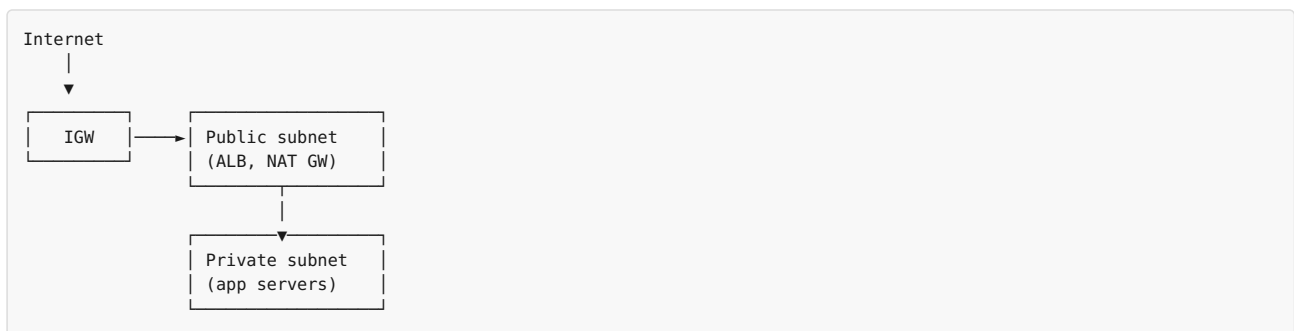
A **route table** defines where traffic goes:

Destination	Target	Meaning
10.0.0.0/16	local	Traffic within VPC
0.0.0.0/0	igw-xxxxx	Internet via Internet Gateway
0.0.0.0/0	nat-xxxxx	Internet via NAT Gateway (private subnet)
pl-xxxxx	VPC Endpoint	Private AWS service access

Each subnet associates with **one** route table.

21.5 9.5 Internet Gateway and NAT (preview)

- **Internet Gateway (IGW)** — VPC component for bidirectional internet access
- **NAT Gateway** — Allows private subnet instances to reach internet (outbound only); place in public subnet



21.6 9.6 Security Groups vs NACLs

	Security Group	NACL
Level	Instance/ENI	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny
Default	Deny all inbound	Allow all

Best practice: Use security groups as primary firewall; NACLs for subnet-level deny rules if needed.

Example security group for web tier:

- Inbound: 443 from ALB security group
- Outbound: 5432 to database security group

21.7 9.7 Terraform VPC snippet

```
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "prod-vpc" }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = true
}
```

21.8 9.8 Chapter summary

- Design VPCs with **public**, **private**, and **database** subnet tiers across **multiple AZs**.
- **Route tables** control internet and local routing.
- **Security groups** are stateful instance firewalls; default deny inbound.

21.9 Lab 9.1

1. Create a custom VPC with 2 public and 2 private subnets in different AZs.
2. Attach an Internet Gateway and configure route tables.
3. Launch an EC2 in a private subnet (no public IP) and verify no direct internet access.

Next: [Chapter 10 — NAT and Routing](#)

22 Chapter 10: Internet Gateway, NAT Gateway, and Routing

AWS Handbook — Part III, Pages 186–205

22.1 10.1 Internet Gateway (IGW)

An **Internet Gateway** is a horizontally scaled, redundant VPC component that allows:

- Instances with public IPs to communicate with the internet
- Internet to initiate connections to instances with public IPs

Attach one IGW per VPC (for internet-connected VPCs).

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id igw-xxx --vpc-id vpc-xxx
```

Public subnet route table:

Destination	Target
0.0.0.0/0	igw-xxxxxxx
10.0.0.0/16	local

22.2 10.2 NAT Gateway

Private subnet instances should **not** have public IPs. They reach the internet (for updates, API calls) via a **NAT Gateway** in a public subnet.

```
Private subnet instance (10.0.10.5)
  → NAT Gateway (public subnet, elastic IP)
  → Internet Gateway
  → Internet
```

22.2.1 NAT Gateway facts

- **Managed** by AWS (no patching)
- **Charged** per hour + per GB processed
- **AZ-specific** — deploy one NAT per AZ for HA, or accept cross-AZ failover tradeoffs
- **Alternative:** NAT instances (self-managed, not recommended)

Private subnet route table:

Destination	Target
0.0.0.0/0	nat-xxxxxxx
10.0.0.0/16	local

22.3 10.3 VPC Endpoints — avoid NAT costs

For AWS service traffic (S3, DynamoDB, etc.), use **VPC endpoints** to keep traffic on the AWS network—no NAT charges.

Type	Services	Cost
Gateway	S3, DynamoDB	Free
Interface	Most other services	Per hour + data

```
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  route_table_ids = [aws_route_table.private.id]
}
```

22.4 10.4 Elastic IP (EIP)

A **static public IPv4** address. Required for NAT Gateway.

- **Charge** applies if EIP is allocated but not attached to a running instance
- Release unused EIPs to avoid fees

22.5 10.5 Troubleshooting connectivity

Problem	Check
Can't SSH to public instance	SG allows 22; has public IP; IGW attached; correct route
Private instance can't reach internet	NAT in public subnet; private RT has 0.0.0.0/0 → NAT
Can't reach S3 from private subnet	VPC endpoint or NAT route
Inter-subnet blocked	NACL rules; SG rules

22.5.1 Useful commands

```
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-xxx"
aws ec2 describe-network-interfaces --filters "Name=subnet-id,Values=subnet-xxx"
# From instance:
curl -s https://checkip.amazonaws.com # Public IP test
```

22.6 10.6 High availability networking

Production checklist:

- ☐ Multi-AZ subnets
- ☐ NAT Gateway per AZ (or accept risk)
- ☐ ALB spans multiple AZs
- ☐ VPC endpoints for S3/DynamoDB/ECR/API
- ☐ No single-AZ dependencies for critical paths

22.7 10.7 Chapter summary

- **IGW** enables public internet for public subnets.

- **NAT Gateway** provides outbound-only internet for private subnets.
 - Use **VPC endpoints** to reduce NAT costs and improve security for AWS APIs.
-

22.8 Lab 10.1

1. Deploy NAT Gateway in public subnet with EIP.
 2. Configure private subnet route table.
 3. Launch EC2 in private subnet; `curl` an external API successfully.
 4. Add S3 gateway endpoint; verify S3 access without NAT.
-

Next: [Chapter 15 — EC2](#)

23 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates

AWS Handbook — Part IV, Pages 281–305

23.1 15.1 Amazon EC2 overview

Elastic Compute Cloud (EC2) provides resizable virtual machines in the cloud. You choose:

- **Instance type** — CPU, memory, network, storage
- **AMI** — Operating system image
- **Networking** — VPC, subnet, security groups
- **Storage** — EBS volumes
- **IAM role** — Permissions without access keys

23.2 15.2 Instance families

Family	Use case	Examples
T (burstable)	Dev, low-traffic web	t3.micro, t3.small
M (general)	Balanced workloads	m6i.large
C (compute)	CPU-intensive	c6i.xlarge
R (memory)	Databases, caches	r6i.large
G/P (GPU)	ML, graphics	g5.xlarge
I (storage)	High IOPS databases	i3.large

Naming: `m6i.large` = family **m**, gen **6**, Intel **i**, size **large**.

23.2.1 Purchasing options

Option	Cost	Interruption
On-Demand	Highest	Never
Reserved / Savings Plans	30–70% less	Never
Spot	Up to 90% less	Can be terminated with 2-min notice
Dedicated Host	Compliance	Never

23.3 15.3 AMIs and launch templates

AMI (Amazon Machine Image) — Template for root volume: OS + optional software.

Sources:

- AWS-provided (Amazon Linux 2023, Ubuntu)
- Marketplace
- Custom (built with EC2 Image Builder or Packer)

Launch Template — Reusable EC2 configuration (instance type, AMI, SG, user data, IAM role). Preferred over launch configurations.

```
aws ec2 run-instances \  
  --image-id ami-0c55b159cbfafelf0 \  
  --instance-type t3.micro \  
  --key-name my-key \  
  --security-group-ids sg-xxx \  
  --subnet-id subnet-xxx \  
  --iam-instance-profile Name=EC2-S3-ReadRole \  
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-01}]'
```

23.4 15.4 User data — bootstrap scripts

Run commands on first boot:

```
#!/bin/bash  
yum update -y  
yum install -y nginx  
systemctl enable nginx  
systemctl start nginx  
echo "<h1>Hello from $(hostname)</h1>" > /usr/share/nginx/html/index.html
```

Pass via console, CLI `--user-data`, or launch template. Logs: `/var/log/cloud-init-output.log`.

23.5 15.5 Auto Scaling Groups (ASG)

ASG maintains desired instance count and replaces unhealthy instances.

```
resource "aws_autoscaling_group" "web" {  
  name                = "web-asg"  
  vpc_zone_identifier = [aws_subnet.public_a.id, aws_subnet.public_b.id]  
  target_group_arns   = [aws_lb_target_group.web.arn]  
  health_check_type   = "ELB"  
  min_size            = 2  
  max_size            = 10  
  desired_capacity    = 2  
  
  launch_template {  
    id       = aws_launch_template.web.id  
    version = "$Latest"  
  }  
}
```

Pair with **ALB** for traffic distribution and **scaling policies** on CPU or request count.

23.6 15.6 EC2 best practices

- Use **IAM roles**, never embed access keys
 - **Session Manager** instead of SSH where possible (no open port 22)
 - **IMDSv2** required (`HttpTokens: required`) — prevents SSRF credential theft
 - **Encrypted EBS** volumes by default
 - **Patch** via SSM Patch Manager or golden AMI pipelines
 - **Right-size** with CloudWatch metrics and Compute Optimizer
-

23.7 15.7 Chapter summary

- Choose instance **family and size** based on workload profile.
 - Use **launch templates** and **user data** for repeatable deployments.
 - **ASG + ALB** provides scalable, self-healing web tiers.
-

23.8 Lab 15.1

1. Launch Amazon Linux 2023 t3.micro with nginx user data.
 2. Attach IAM role for S3 read.
 3. Create launch template and ASG (min 1, max 2) behind an ALB.
-

Next: [Chapter 20 — S3](#)

24 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle

AWS Handbook — Part V, Pages 401–425

24.1 20.1 Amazon S3 overview

Simple Storage Service (S3) is object storage: store files (objects) in containers (buckets). Virtually unlimited scale; 99.999999999% durability.

Use cases:

- Static websites and assets
- Data lakes and backups
- Application uploads
- Log archives
- Terraform state files

24.2 20.2 Core concepts

Concept	Description
Bucket	Global unique name; lives in a region
Object	File + metadata; up to 5 TB
Key	Object path (e.g., <code>images/photo.jpg</code>)
Prefix	Simulated folders via key prefixes

```
aws s3 mb s3://my-unique-bucket-name-12345
aws s3 cp report.pdf s3://my-bucket/reports/2026/
aws s3 ls s3://my-bucket/reports/
aws s3 sync ./local-dir s3://my-bucket/backup/
aws s3 rm s3://my-bucket/old/ --recursive
```

24.3 20.3 Storage classes

Class	Use case	Retrieval
Standard	Frequent access	Immediate
Standard-IA	Infrequent access	Immediate, min storage charge
Glacier Instant	Archive, instant retrieval	Immediate
Glacier Flexible	Archive	Minutes to hours
Glacier Deep Archive	Long-term archive	12–48 hours
Intelligent-Tiering	Unknown access patterns	Auto-moves tiers

24.4 20.4 Versioning and lifecycle

24.4.1 Versioning

Protects against accidental deletes; keeps all versions of objects.

```
aws s3api put-bucket-versioning \
  --bucket my-bucket \
  --versioning-configuration Status=Enabled
```

24.4.2 Lifecycle rules

Automatically transition or expire objects:

```
{
  "Rules": [{
    "ID": "archive-old-logs",
    "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [{
      "Days": 30,
      "StorageClass": "STANDARD_IA"
    }, {
      "Days": 90,
      "StorageClass": "GLACIER"
    }],
    "Expiration": { "Days": 365 }
  }]
}
```

24.5 20.5 Security

24.5.1 Block Public Access (account and bucket level)

Enable by default on all buckets unless hosting a public static site intentionally.

24.5.2 Encryption

Method	Description
SSE-S3	AWS-managed keys
SSE-KMS	KMS keys; audit trail
SSE-C	Customer-provided keys
Client-side	Encrypt before upload

24.5.3 Bucket policies vs ACLs

- **Bucket policies** (JSON) — Preferred; resource-based IAM
- **ACLs** — Legacy; avoid for new projects

Example: allow CloudFront OAC to read objects (not public internet).

24.6 20.6 Static website hosting

```
# Enable static website hosting on bucket
aws s3 website s3://my-bucket/ --index-document index.html
```

Front with **CloudFront** + **ACM certificate** for HTTPS. Do not expose bucket directly to public.

24.7 20.7 S3 performance tips

- Use **multipart upload** for files > 100 MB
 - Prefix keys for parallelism (S3 scales automatically)
 - **S3 Transfer Acceleration** for long-distance uploads
 - **S3 Inventory** for auditing object metadata
-

24.8 20.8 Chapter summary

- S3 stores **objects in buckets** with global-unique names.
 - Enable **versioning** and **lifecycle** for data protection and cost control.
 - **Block public access**; use bucket policies and CloudFront for controlled delivery.
-

24.9 Lab 20.1

1. Create bucket with versioning and default encryption (SSE-S3).
 2. Upload files; delete one; restore previous version.
 3. Add lifecycle rule to transition `logs/` to IA after 30 days.
 4. Host static `index.html` behind CloudFront.
-

Next: [Chapter 35 — CloudWatch](#)

25 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards

AWS Handbook — Part IX, Pages 701–720

25.1 35.1 Amazon CloudWatch overview

CloudWatch is AWS’s observability service for **metrics**, **logs**, **alarms**, and **dashboards**. Nearly every AWS service emits CloudWatch metrics automatically.

25.2 35.2 Metrics

A **metric** is a time-series datapoint (e.g., `CPUUtilization` for EC2).

Concept	Description
Namespace	Grouping (AWS/EC2, AWS/S3, custom)
Metric name	e.g., <code>CPUUtilization</code>
Dimensions	InstanceId, FunctionName, etc.
Resolution	Standard 1 min; high-resolution down to 1 sec

25.2.1 View metrics

```
aws cloudwatch list-metrics --namespace AWS/EC2
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --start-time 2026-09-01T00:00:00Z \
  --end-time 2026-09-01T12:00:00Z \
  --period 300 \
  --statistics Average
```

25.2.2 Custom metrics

```
import boto3
cloudwatch = boto3.client('cloudwatch')
cloudwatch.put_metric_data(
    Namespace='MyApp/Orders',
    MetricData=[{
        'MetricName': 'OrdersProcessed',
        'Value': 42,
        'Unit': 'Count'
    }]
)
```

25.3 35.3 Alarms

Alarms trigger actions when metrics breach thresholds.

```
aws cloudwatch put-metric-alarm \
```

```
--alarm-name high-cpu-web-server \
--metric-name CPUUtilization \
--namespace AWS/EC2 \
--statistic Average \
--period 300 \
--threshold 80 \
--comparison-operator GreaterThanThreshold \
--evaluation-periods 2 \
--alarm-actions arn:aws:sns:us-east-1:123456789012:ops-alerts \
--dimensions Name=InstanceId,Value=i-0abc123
```

25.3.1 Alarm states

State	Meaning
OK	Within threshold
ALARM	Breached threshold
INSUFFICIENT_DATA	Not enough datapoints

25.3.2 Composite alarms

Combine multiple alarms with AND/OR logic to reduce alert noise.

25.4 35.4 Dashboards

Visualize metrics in the console or via API. Example Terraform:

```
resource "aws_cloudwatch_dashboard" "main" {
  dashboard_name = "production-overview"
  dashboard_body = jsonencode({
    widgets = [{
      type = "metric"
      width = 12
      height = 6
      properties = {
        metrics = [
          ["AWS/EC2", "CPUUtilization", "InstanceId", "i-0abc123"],
          [".", "NetworkIn", ".", "."]
        ]
        period = 300
        stat = "Average"
        region = "us-east-1"
        title = "EC2 Web Server"
      }
    }]
  })
}
```

25.5 35.5 CloudWatch Logs (overview)

Feature	Purpose
Log groups / streams	Organize application logs
Logs Insights	SQL-like query language
Subscription filters	Stream to Lambda, Kinesis, OpenSearch
Metric filters	Turn log patterns into metrics

```
aws logs tail /aws/lambda/my-function --follow
```

25.6 35.6 Observability best practices on AWS

1. **Alarm on symptoms** — Error rate, latency p99, not just CPU
 2. **SNS → PagerDuty/Slack** for on-call routing
 3. **Dashboards per service** + one executive overview
 4. **Log retention** — Set retention policies (cost control)
 5. **X-Ray** for distributed tracing (complements CloudWatch)
 6. Use **Container Insights** / **Lambda Insights** for managed runtimes
-

25.7 35.7 Chapter summary

- CloudWatch collects **metrics** from AWS and custom apps.
 - **Alarms** notify via SNS when thresholds breach.
 - **Dashboards** and **Logs** complete the operational picture.
-

25.8 Lab 35.1

1. Create SNS topic `ops-alerts` (email subscription).
 2. Create CPU alarm on an EC2 instance.
 3. Build a dashboard with EC2 CPU, ALB request count, and RDS connections.
 4. Run a Logs Insights query on a Lambda log group.
-

Continue: Chapter 36 — CloudWatch Logs Deep Dive (outline in TOC)